

TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH

PHÁT HIỆN TIN GIẢ DỰA TRÊN CÁC KỸ THUẬT TRÍ TUỆ NHÂN TẠO

Báo cáo Nghiên cứu Khoa học Sinh viên

Sinh viên thực hiện: Nguyễn Hoàng Anh
Giảng viên hướng dẫn: Dương Hữu Thành
Năm học: 2025–2026

Thành phố Hồ Chí Minh, tháng 02 năm 2026

Mục lục

Danh sách bảng	1
Danh sách hình vẽ	2
Danh sách từ viết tắt	3
Danh sách thuật ngữ tiếng Anh	5
Danh sách ký hiệu	8
Tóm tắt đề tài	9
Tổng quan về đề tài	10
1 Cơ sở lý thuyết	12
1.1 Một số khái niệm cơ bản	12
1.1.1 Văn bản và biểu diễn văn bản	12
1.1.2 Đặc trưng văn bản và tiền xử lý	12
1.1.3 Bài toán phân loại văn bản	13
1.2 Xử lý ngôn ngữ tự nhiên tiếng Việt	13
1.2.1 Đặc điểm ngôn ngữ tiếng Việt	13
1.2.2 Phân đoạn từ (Word Segmentation)	14
1.2.3 Biểu diễn văn bản số	14
1.3 Học máy	14
1.3.1 Định nghĩa	14
1.3.2 Một số thuật toán học máy tiêu biểu	15
1.3.2.1 Hồi quy Logistic (Logistic Regression – LR)	15
1.3.2.2 Máy vectơ hỗ trợ (Support Vector Machine – SVM)	15
1.3.3 Biểu diễn đặc trưng cơ bản	16
1.3.3.1 Tần suất từ – Tần suất tài liệu nghịch đảo (TF-IDF)	16
1.4 Học sâu	17
1.4.1 Định nghĩa	17

1.4.2	Mạng nơ-ron nhân tạo (Artificial Neural Network)	18
1.4.2.1	Một số thành phần cần chú ý	19
1.4.3	Mạng nơ-ron hồi quy (Recurrent Neural Network – RNN) . . .	19
1.4.4	Bộ nhớ dài-ngắn hạn (Long Short-Term Memory – LSTM) . . .	20
1.4.5	Mạng LSTM hai chiều (Bidirectional LSTM)	22
1.4.6	Cơ chế chú ý (Attention Mechanism)	23
1.4.7	Mô hình Transformer và PhoBERT	24
1.5	Kết luận chương	26
2	Bộ dữ liệu và tiền xử lý	27
2.1	Thu thập và mô tả bộ dữ liệu	27
2.1.1	Nguồn dữ liệu	27
2.1.2	Phân phối nhãn	27
2.2	Tiền xử lý văn bản	28
2.2.1	Làm sạch và lọc dữ liệu	28
2.2.2	Phân đoạn từ bằng VnCoreNLP	29
2.3	Chia tập dữ liệu	30
2.3.1	Chia phân tầng	30
2.3.2	Kiểm tra rò rỉ dữ liệu	32
2.4	Kết luận chương	32
3	Xác định tin giả	34
3.1	Phương pháp đề xuất	34
3.2	Lý do lựa chọn các nhóm mô hình	34
3.3	Tổng quan kiến trúc hệ thống	36
3.4	Trích xuất đặc trưng	36
3.4.1	TF-IDF cho mô hình học máy truyền thống	36
3.4.1.1	Lý do lựa chọn TF-IDF	36
3.4.1.2	Cấu hình TF-IDF	37
3.4.2	Nhúng từ (Word Embedding) cho BiLSTM	37
3.4.3	Tokenization PhoBERT (BPE)	38
3.5	Các mô hình phân loại	38
3.5.1	Các mô hình học máy truyền thống	38
3.5.1.1	Hồi quy Logistic	38
3.5.1.2	Máy vectơ hỗ trợ	39
3.5.2	Các mô hình học sâu	39
3.5.2.1	Mạng BiLSTM	39
3.5.2.2	Tinh chỉnh PhoBERT	40

3.6	Chiến lược xử lý mất cân bằng nhãn	40
3.7	Kết luận chương	41
4	Kết quả thực nghiệm	42
4.1	Môi trường thực nghiệm	42
4.2	So sánh hiệu năng các mô hình	43
4.2.1	Kết quả tổng thể	43
4.2.2	Hiệu năng theo từng lớp	44
4.2.3	Ma trận nhầm lẫn	45
4.2.4	Đường cong ROC và Precision-Recall	46
4.2.5	Phân tích hiệu chuẩn xác suất (Calibration Analysis)	46
4.3	Kiểm định thống kê	49
4.3.1	Kiểm định McNemar	49
4.3.2	Khoảng tin cậy Bootstrap	50
4.3.3	Kiểm tra chéo (Cross-Validation)	52
4.4	Nghiên cứu triệt bỏ (Ablation Study)	53
4.5	Phân tích lỗi	57
4.5.1	Thống kê lỗi tổng quan	57
4.5.1.1	So sánh FP và FN	57
4.5.1.2	Phân phối lỗi theo độ dài văn bản	58
4.5.1.3	Hàm ý cải tiến mô hình	59
4.5.1.4	Phân tích các trường hợp khó (Hard Examples)	59
4.5.2	Phân tích khả năng giải thích	61
4.6	Kết luận chương	62
	Kết luận	63

Danh sách bảng

2.1	Thống kê tập dữ liệu	31
4.1	Siêu tham số các mô hình	42
4.2	Độ phức tạp và thời gian huấn luyện	42
4.3	So sánh hiệu suất các mô hình trên tập kiểm tra	43
4.4	Chỉ số hiệu suất theo từng lớp	44
4.5	Chỉ số hiệu chuẩn xác suất của bốn mô hình trên tập kiểm tra	48
4.6	Kết quả kiểm định McNemar (sau hiệu chỉnh Holm-Bonferroni)	50
4.7	Khoảng tin cậy 95% Bootstrap [1] cho F1-Score (10,000 lần lấy mẫu lại)	51
4.8	Kết quả kiểm tra chéo 5-fold (3 seeds)	52
4.9	Kết quả Ablation Study trên tập kiểm tra	55
4.10	Phân tích lỗi trên tập kiểm tra	59

Danh sách hình vẽ

1.1	Các bước tiền xử lý văn bản trước khi đưa vào mô hình học máy. . . .	13
1.2	Minh họa thuật toán nhị phân SVM (phỏng theo [2])	16
1.3	Minh họa cấu trúc mạng LSTM [3].	21
1.4	Minh họa cấu trúc mạng LSTM hai chiều [4].	23
1.5	Minh họa khối encoder của Transformer [5].	25
2.1	Phân phối nhãn theo từng tập dữ liệu và tổng quan toàn bộ 15,789 mẫu	27
2.2	Phân phối nhãn trong các tập dữ liệu huấn luyện, kiểm định và kiểm tra	31
4.1	So sánh các chỉ số hiệu năng (Accuracy, F1-Score, ROC-AUC) của bốn mô hình trên tập kiểm tra	43
4.2	Precision, Recall và F1-Score theo từng lớp (Thật / Giả) cho bốn mô hình	44
4.3	Ma trận nhầm lẫn của bốn mô hình trên tập kiểm tra (2,094 mẫu) . . .	45
4.4	Đường cong ROC và Precision-Recall của bốn mô hình trên tập kiểm tra	46
4.5	Biểu đồ hiệu chuẩn (Reliability Diagram) của bốn mô hình trên tập kiểm tra. Đường chéo nét đứt biểu diễn hiệu chuẩn hoàn hảo; các điểm vuông biểu diễn tỷ lệ dương tính thực tế trong mỗi bin xác suất.	48
4.6	So sánh số lượng False Positive (FP) và False Negative (FN) trong quá trình phân loại	57
4.7	Phân phối lỗi phân loại theo độ dài văn bản	58
4.8	Top 15 đặc trưng TF-IDF có hệ số LR cao nhất (dương = chỉ điểm Giả, âm = chỉ điểm Thật)	61
4.9	Phân loại lỗi theo độ dài văn bản và mức độ tin cậy dự đoán	62

Danh sách từ viết tắt

Từ viết tắt	Giải nghĩa
AI	Artificial Intelligence – Trí tuệ nhân tạo
AUC	Area Under the Curve – Diện tích dưới đường cong ROC
BERT	Bidirectional Encoder Representations from Transformers
BiLSTM	Bidirectional Long Short-Term Memory – Mạng LSTM hai chiều
BPE	Byte Pair Encoding – Mã hoá cặp byte
CI	Confidence Interval – Khoảng tin cậy
CUDA	Compute Unified Device Architecture
CV	Cross-Validation – Kiểm tra chéo
F1	F1-Score – Trung bình điều hoà Precision và Recall
FFN	Feed-Forward Network – Mạng truyền thẳng
FN	False Negative – Âm tính giả
FP	False Positive – Dương tính giả
GPU	Graphics Processing Unit – Bộ xử lý đồ hoạ
IDF	Inverse Document Frequency – Tần số nghịch đảo tài liệu
LR	Logistic Regression – Hồi quy Logistic
LSTM	Long Short-Term Memory
ML	Machine Learning – Học máy
NLP	Natural Language Processing – Xử lý ngôn ngữ tự nhiên
PhoBERT	Pretrained Language Model for Vietnamese (VinAI Research)
ReLU	Rectified Linear Unit – Hàm kích hoạt tuyến tính chỉnh lưu
RNN	Recurrent Neural Network – Mạng nơ-ron hồi tiếp
ROC	Receiver Operating Characteristic
SVM	Support Vector Machine – Máy vectơ hỗ trợ
TF	Term Frequency – Tần số xuất hiện của từ
TF-IDF	Term Frequency – Inverse Document Frequency

TN	True Negative – Âm tính thật
TP	True Positive – Dương tính thật
ECE	Expected Calibration Error – Sai số hiệu chuẩn kỳ vọng
MCE	Maximum Calibration Error – Sai số hiệu chuẩn cực đại

Danh sách thuật ngữ tiếng Anh

Thuật ngữ tiếng Anh	Giải thích
Ablation Study	Nghiên cứu triệt bỏ từng thành phần để đo đóng góp
Attention Mechanism	Cơ chế chú ý cho phép mô hình tập trung vào phần quan trọng
Backpropagation	Lan truyền ngược – thuật toán cập nhật trọng số dựa trên gradient
Batch Size	Số mẫu xử lý đồng thời trong mỗi bước huấn luyện
Bootstrap	Phương pháp lấy mẫu lại có hoàn lại để ước lượng phân phối thống kê
Class Weighting	Trọng số lớp – tăng phạt cho lớp thiểu số trong hàm mất mát
Cross-Entropy	Hàm mất mát đo sự khác biệt giữa phân phối dự đoán và thực tế
Cross-Validation	Kiểm tra chéo – đánh giá mô hình trên nhiều lần chia dữ liệu
Data Leakage	Rò rỉ dữ liệu – thông tin test set lọt vào quá trình huấn luyện
Dropout	Kỹ thuật tắt ngẫu nhiên neuron trong huấn luyện để giảm quá khớp
Early Stopping	Dừng sớm – dừng huấn luyện khi hiệu năng val không cải thiện
Embedding	Biểu diễn từ bằng vectơ số thực nhiều chiều
Fine-tuning	Tinh chỉnh – cập nhật nhẹ nhàng mô hình đã được tiền huấn luyện
Gradient Clipping	Cắt gradient – giới hạn biên độ gradient để ổn định huấn luyện
Grid Search	Tìm kiếm lưới – duyệt tổ hợp siêu tham số để chọn cấu hình tốt nhất
Hyperparameter	Siêu tham số – tham số đặt trước khi huấn luyện

Learning Rate	Tốc độ học – hệ số điều chỉnh bước cập nhật trọng số
McNemar’s Test	Kiểm định thống kê so sánh hai bộ phân loại theo từng mẫu
Overfitting	Quá khớp – mô hình học thuộc dữ liệu train, kém tổng quát
Precision	Độ chính xác dương – tỷ lệ dự đoán dương đúng trên tổng dự đoán dương
Recall	Độ thu hồi – tỷ lệ mẫu dương được phát hiện đúng
Stratified Split	Chia phân tầng – giữ nguyên tỉ lệ nhãn trong mỗi tập
Subword	Đơn vị con của từ – thành phần nhỏ hơn từ dùng trong BPE
Tokenization	Tách chuỗi ký tự thành các đơn vị (token)
Transformer	Kiến trúc mạng nơ-ron dùng cơ chế tự chú ý (self-attention)
Warmup Schedule	Lịch khởi động – tăng dần tốc độ học ở đầu quá trình huấn luyện
Word Segmentation	Phân đoạn từ – tách văn bản tiếng Việt thành các từ
Calibration	Hiệu chuẩn xác suất – đánh giá tương quan giữa xác suất dự đoán và kết quả thực tế
Brier Score	Trung bình bình phương sai số giữa xác suất dự đoán và nhãn thực
Reliability Diagram	Biểu đồ hiệu chuẩn – trực quan hóa chất lượng xác suất dự đoán
Bag-of-Words	Biểu diễn văn bản theo tần suất xuất hiện từ, bỏ qua thứ tự
Baseline	Mô hình cơ sở dùng làm điểm so sánh cho các phương pháp khác
Domain Shift	Dịch chuyển miền – sự khác biệt phân phối giữa dữ liệu huấn luyện và triển khai
Fact-checking	Kiểm chứng thực tế – xác minh tính chính xác của thông tin
Knowledge Distillation	Chưng cất tri thức – chuyển giao kiến thức từ mô hình lớn sang mô hình nhỏ

Pipeline	Quy trình xử lý dữ liệu gồm nhiều bước nối tiếp nhau
Temperature Scaling	Kỹ thuật hiệu chuẩn hậu xử lý điều chỉnh độ tin cậy xác suất dự đoán

Danh sách ký hiệu

Ký hiệu	Ý nghĩa
x_i	Văn bản đầu vào thứ i
y_i	Nhãn thực thứ i ($0 = \text{Thật}$, $1 = \text{Giả}$)
$\hat{y}_i$	Nhãn dự đoán thứ i
N	Tổng số văn bản trong tập dữ liệu
V	Kích thước từ điển (vocabulary size)
d	Số chiều của vectơ biểu diễn
h_t	Trạng thái ẩn tại bước thời gian t
$\mathbf{W}$	Ma trận trọng số
$\mathbf{b}$	Vectơ độ lệch (bias)
σ	Hàm kích hoạt sigmoid
α_t	Trọng số chú ý tại bước thời gian t
C	Tham số điều chuẩn (regularization) của SVM và LR
η	Tốc độ học (learning rate)
λ	Hệ số phân rã trọng số (weight decay)

Tóm tắt đề tài

Tin giả trên các nền tảng số tiếng Việt đang đặt ra thách thức cấp bách cho các nhà nghiên cứu NLP, do đặc điểm hình thái học của tiếng Việt – ngôn ngữ đơn lập, không biến hình – vốn đòi hỏi những kỹ thuật tiền xử lý và biểu diễn đặc trưng riêng biệt. Trong khi nhiều nghiên cứu quốc tế đã đạt kết quả đáng chú ý trên dữ liệu tiếng Anh, hiệu quả của các phương pháp tương tự trên tiếng Việt chưa được khảo sát đầy đủ.

Nghiên cứu này đề xuất và so sánh bốn phương pháp phân loại tin tức Việt Nam: Hồi quy Logistic (LR) và Máy vectơ hỗ trợ (SVM) sử dụng đặc trưng tần suất từ – tần suất tài liệu nghịch đảo (Term Frequency – Inverse Document Frequency, TF-IDF), Mạng LSTM hai chiều (BiLSTM) với cơ chế chú ý, và mô hình ngôn ngữ PhoBERT tinh chỉnh theo tác vụ. Thực nghiệm được thực hiện trên bộ dữ liệu gồm 13,958 mẫu sau khi làm sạch, được chia theo tỉ lệ 70/15/15 (huấn luyện/kiểm định/kiểm tra).

Kết quả cho thấy PhoBERT đạt hiệu năng tốt nhất ($F1 = 89.88\%$, $ROC-AUC = 0.9495$), vượt trội rõ rệt so với LR (83.29%), SVM (84.10%) và BiLSTM (82.32%). Đáng chú ý, SVM lại vượt BiLSTM bất chấp kiến trúc đơn giản hơn nhiều, cho thấy chất lượng biểu diễn đặc trưng có thể bù đắp cho độ phức tạp của mô hình khi dữ liệu có giới hạn. Kiểm định McNemar với hiệu chỉnh Holm-Bonferroni xác nhận sự vượt trội của PhoBERT có ý nghĩa thống kê ($p < 0.001$). Nghiên cứu ablation cho thấy tiền huấn luyện trên ngữ liệu tiếng Việt quy mô lớn là nhân tố quyết định; phân đoạn từ và kích thước từ điển chỉ đóng vai trò thứ yếu đối với các mô hình học máy truyền thống.

Từ khoá: phát hiện tin giả, tiếng Việt, PhoBERT, BiLSTM, TF-IDF, xử lý ngôn ngữ tự nhiên.

Tổng quan về đề tài

1. Đặt vấn đề

Mạng xã hội và các nền tảng tin tức trực tuyến tại Việt Nam đã rút ngắn tối thiểu khoảng cách giữa sự kiện và công chúng, nhưng cũng đồng thời tạo ra môi trường thuận lợi cho tin sai lệch phát tán rộng rãi trước khi được kiểm chứng. Theo thống kê, Việt Nam nằm trong nhóm các quốc gia có tỉ lệ tiếp xúc với tin giả cao nhất tại Đông Nam Á. Việc kiểm chứng thủ công không thể theo kịp quy mô sản xuất và lưu truyền nội dung hiện nay; đó là lý do các hệ thống phát hiện tin giả tự động bằng học máy và học sâu trở nên cấp thiết.

2. Mục tiêu nghiên cứu

- Xây dựng và so sánh bốn mô hình phân loại tin tức tiếng Việt: LR, SVM, BiLSTM và PhoBERT.
- Đánh giá ảnh hưởng của phân đoạn từ tiếng Việt đối với hiệu năng mô hình.
- Thực hiện đánh giá toàn diện bao gồm kiểm định thống kê, kiểm tra chéo, và phân tích giải thích.
- Xác định phương pháp tốt nhất cho bài toán phát hiện tin giả tiếng Việt.

3. Phạm vi nghiên cứu

Nghiên cứu tập trung vào phân loại nhị phân văn bản tiếng Việt (Thật/Giả) dựa trên tiêu đề và nội dung bài viết. Dữ liệu được thu thập từ các bài đăng công khai trên mạng xã hội (Facebook) và bào điện tử trong giai đoạn COVID-19, bao gồm ngôn ngữ tự nhiên, không được kiểm soát

4. Bố cục báo cáo

Báo cáo gồm bốn chương chính:

- **Chương 1** trình bày cơ sở lý thuyết về xử lý ngôn ngữ tự nhiên tiếng Việt, các phương pháp học máy và kiến trúc mô hình học sâu được sử dụng.
- **Chương 2** mô tả bộ dữ liệu, quy trình tiền xử lý và chia tập.
- **Chương 3** trình bày chi tiết các phương pháp đề xuất, kiến trúc mô hình và quy trình huấn luyện.
- **Chương 4** trình bày kết quả thực nghiệm, phân tích lỗi, kiểm định thống kê và nghiên cứu ablation.

Chương 1 Cơ sở lý thuyết

Chương này thiết lập nền tảng lý thuyết cho bốn mô hình được khảo sát trong nghiên cứu. Phần đầu làm rõ các khái niệm biểu diễn văn bản và đặc thù ngôn ngữ tiếng Việt; phần tiếp theo phân tích nguyên lý hoạt động của hai thuật toán học máy truyền thống – LR và SVM. Phần cuối đi sâu vào các kiến trúc học sâu: từ LSTM tuần tự, BiLSTM hai chiều, đến Transformer song song, qua đó lý giải vì sao PhoBERT – nhờ cơ chế self-attention nắm bắt ngữ cảnh toàn cục – đạt hiệu năng vượt trội trong bài toán phát hiện tin giả tiếng Việt.

1.1 Một số khái niệm cơ bản

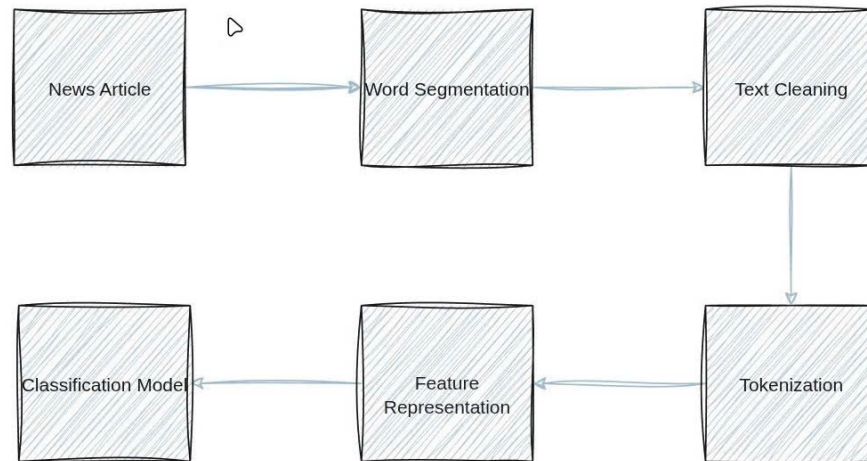
1.1.1 Văn bản và biểu diễn văn bản

Mô hình học máy không xử lý trực tiếp chuỗi ký tự mà yêu cầu văn bản phải được chuyển sang dạng số trong một không gian đặc trưng xác định. Hai trường phái biểu diễn chính đối lập nhau về bản chất: Bag-of-Words và TF-IDF ánh xạ mỗi tài liệu thành một vectơ thưa (*sparse vector*) trong không gian từ vựng, hoàn toàn bỏ qua thứ tự từ; ngược lại, các phương pháp embedding chiếu từng từ vào một không gian dày (*dense*) có số chiều thấp, nơi quan hệ ngữ nghĩa được mã hóa ngầm qua cấu trúc hình học của các vectơ. Sự lựa chọn giữa hai trường phái này gắn chặt với kiến trúc mô hình: LR và SVM vận hành hiệu quả trên vectơ TF-IDF thưa nhờ khả năng tối ưu trong không gian chiều cao, còn BiLSTM và PhoBERT đòi hỏi biểu diễn embedding liên tục để khai thác ngữ cảnh tuần tự hoặc toàn cục.

1.1.2 Đặc trưng văn bản và tiền xử lý

Tiền xử lý biến đổi văn bản thô thành đầu vào đồng nhất cho mô hình thông qua chuẩn hóa chữ viết, loại bỏ ký tự nhiễu, tách từ và lọc từ dừng. Đối với tiếng Việt, *phân đoạn từ* (*word segmentation*) đóng vai trò then chốt hơn hẳn so với các ngôn ngữ biến hình: khoảng trắng trong tiếng Việt phân tách âm tiết chứ không phân tách từ, nên nhiều đơn vị từ vựng bao gồm hai hoặc nhiều âm tiết liền kề (“kiến_trúc”, “tiền_huấn_luyện”). Bỏ qua bước này khiến từ vựng phân mảnh nghiêm trọng, đồng thời làm giảm chất lượng vectơ TF-IDF và gây bất tương thích giữa dữ liệu đầu vào với từ điển tiền huấn luyện của PhoBERT.

Test preprocessing pipeline



Hình 1.1: Các bước tiền xử lý văn bản trước khi đưa vào mô hình học máy.

1.1.3 Bài toán phân loại văn bản

Phát hiện tin giả về bản chất là bài toán phân loại nhị phân: mỗi văn bản được gán nhãn “thật” hoặc “giả”. Mô hình cần học một hàm ánh xạ từ không gian đặc trưng của văn bản sang tập nhãn, sao cho ranh giới quyết định phân tách tốt nhất hai lớp trên dữ liệu chưa quan sát.

1.2 Xử lý ngôn ngữ tự nhiên tiếng Việt

1.2.1 Đặc điểm ngôn ngữ tiếng Việt

Tiếng Việt là ngôn ngữ đơn lập, không biến hình, với các đặc điểm khác biệt so với tiếng Anh và nhiều ngôn ngữ châu Âu:

- **Không có khoảng cách giữa các âm tiết:** Ranh giới từ và âm tiết trong tiếng Việt không trùng nhau. Ví dụ, “thành phố” là một từ nhưng được viết thành hai âm tiết riêng biệt.
- **Từ ghép và từ láy:** Nhiều khái niệm được biểu đạt bằng tổ hợp nhiều âm tiết, ví dụ từ ghép “học_sinh”, “kinh_tế”, hay từ láy “lung_linh”.
- **Thanh điệu:** Tiếng Việt có 6 thanh điệu, ảnh hưởng quan trọng đến ngữ nghĩa nhưng không ảnh hưởng đến cấu trúc hình thái học.

- **Ngôn ngữ cô lập:** Quan hệ ngữ pháp được biểu đạt qua trật tự từ và các hư từ, không qua biến đổi hình thái.

Những đặc điểm này đòi hỏi bước tiền xử lý đặc thù – đặc biệt là *phân đoạn từ* – trước khi áp dụng các mô hình học máy.

1.2.2 Phân đoạn từ (Word Segmentation)

Phân đoạn từ là bước xác định ranh giới các từ trong câu tiếng Việt. Hầu hết các mô hình xử lý ngôn ngữ tự nhiên (Natural Language Processing – NLP), đặc biệt là PhoBERT, được huấn luyện trên dữ liệu đã được phân đoạn.

Ví dụ:

Trước: “Thành phố Hồ Chí Minh là trung tâm kinh tế lớn nhất Việt Nam.”

Sau: “Thành_phố Hồ_Chí_Minh là trung_tâm kinh_tế lớn nhất Việt_Nam.”

Trong nghiên cứu này, nhóm sử dụng **VnCoreNLP RDRSegmenter** [6] thông qua thư viện `py_vncorenlp`. Đây là bộ phân đoạn cùng với bộ tokenizer mà PhoBERT được tiền huấn luyện, đảm bảo sự nhất quán về từ vựng.

1.2.3 Biểu diễn văn bản số

Các mô hình học máy cần chuyển đổi văn bản thô sang dạng số. Nghiên cứu này sử dụng ba phương pháp:

1. **TF-IDF:** Vectơ đặc trưng dựa trên tần suất từ, phù hợp cho LR và SVM.
2. **Nhúng từ (Word Embedding):** Vectơ dày đặc học được [7], khởi tạo từ FastText tiếng Việt [8], phù hợp cho BiLSTM.
3. **Mã hóa cặp byte (Byte-Pair Encoding – BPE):** Tách từ thành các đơn vị con, dùng cho PhoBERT.

1.3 Học máy

1.3.1 Định nghĩa

Với sự phát triển của khoa học công nghệ, các ứng dụng tự động ngày càng trở nên phổ biến hơn trong đời sống chúng ta, một phần là nhờ vào các thuật toán học máy. Học máy (Machine Learning – ML) là một nhánh của lĩnh vực trí tuệ nhân tạo

(Artificial Intelligence – AI), chuyên nghiên cứu và phát minh các thuật toán cho phép máy tính có khả năng tự vận hành một công việc nhất định bằng các bộ dữ liệu đã có mà không cần can thiệp nhiều từ lập trình viên. Các thuật toán máy học thường được chia thành 2 loại chính, đó là:

Học có giám sát (Supervised Learning) gồm các thuật toán giúp máy tính xây dựng một hàm dựa trên dữ liệu có sẵn và đã được gán nhãn để có thể xác định nhãn của một dữ liệu mới.

Học không giám sát (Unsupervised Learning) gồm các thuật toán cho phép máy tính học từ các dữ liệu có sẵn nhưng chưa được gán nhãn. Từ đó, phân các dữ liệu này thành các cụm theo yêu cầu.

1.3.2 Một số thuật toán học máy tiêu biểu

1.3.2.1 Hồi quy Logistic (Logistic Regression – LR)

Hồi quy Logistic (Logistic Regression) [9] là một thuật toán học máy có giám sát được sử dụng phổ biến trong các bài toán phân lớp nhị phân. Thuật toán này mô hình hóa mối quan hệ giữa các đặc trưng đầu vào và xác suất thuộc về một lớp thông qua một hàm logistic (sigmoid). Cụ thể, dữ liệu huấn luyện được biểu diễn dưới dạng các vector đặc trưng trong không gian nhiều chiều. Thuật toán sẽ học các tham số của mô hình sao cho xác suất dự đoán phù hợp nhất với nhãn thực tế của dữ liệu huấn luyện.

Khi có một điểm dữ liệu mới (chưa có nhãn), mô hình sẽ tính toán giá trị tuyến tính của điểm đó dựa trên các trọng số đã học, sau đó đưa kết quả qua hàm sigmoid để thu được một giá trị xác suất trong khoảng từ 0 đến 1. Dựa trên một ngưỡng phân loại xác định trước (thường là 0.5), điểm dữ liệu mới sẽ được gán vào một trong hai lớp tương ứng. Nếu xác suất dự đoán lớn hơn hoặc bằng ngưỡng, điểm dữ liệu sẽ được gán vào lớp thứ nhất; ngược lại, nó sẽ được gán vào lớp còn lại. Nhờ cơ chế này, Logistic Regression cho phép thực hiện phân lớp một cách hiệu quả và dễ diễn giải trong nhiều bài toán thực tế.

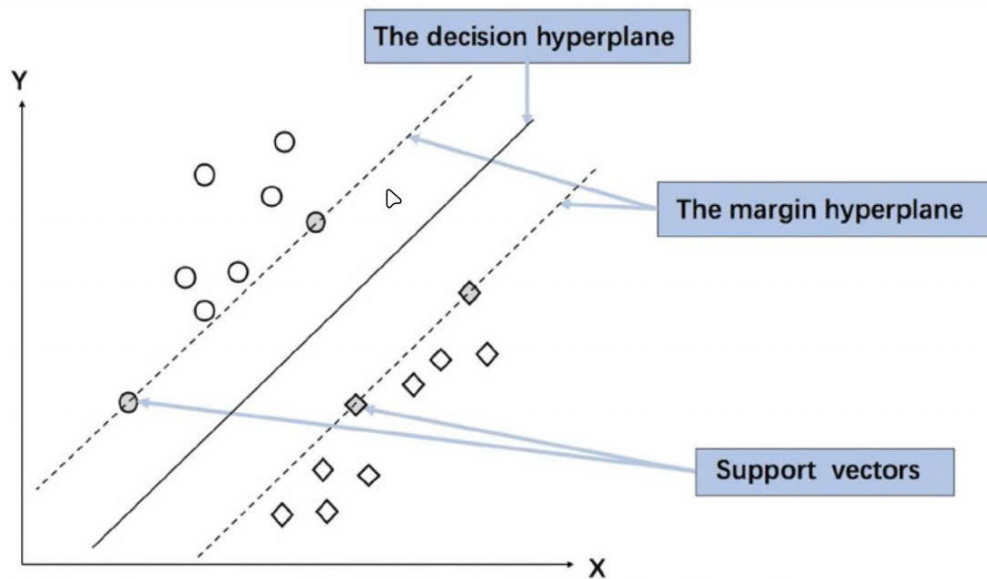
1.3.2.2 Máy vectơ hỗ trợ (Support Vector Machine – SVM)

Máy vectơ hỗ trợ (Support Vector Machine – SVM) [2] là một thuật toán học máy có giám sát được sử dụng phổ biến trong các bài toán phân lớp nhị phân. Đối với bài toán phát hiện tin giả, SVM được sử dụng để phân loại văn bản thành hai lớp là “thật” và “giả”. Thuật toán hoạt động dựa trên việc tìm một siêu phẳng (hyperplane) trong không gian đặc trưng sao cho có thể phân tách hai lớp dữ liệu một cách tối ưu.

Cụ thể, các dữ liệu huấn luyện sau khi được biểu diễn dưới dạng vector đặc trưng (ví dụ như TF-IDF) sẽ được ánh xạ vào không gian nhiều chiều. Trong không gian này,

SVM tìm một siêu phẳng sao cho khoảng cách từ siêu phẳng đến các điểm gần nhất của mỗi lớp là lớn nhất. Khoảng cách này được gọi là biên (margin). Những điểm dữ liệu nằm gần siêu phẳng nhất và quyết định vị trí của nó được gọi là các vector hỗ trợ (support vectors).

Khi đã xác định được siêu phẳng tối ưu, với một điểm dữ liệu mới, mô hình sẽ xác định vị trí của điểm đó so với siêu phẳng để gán nhãn tương ứng. Nếu điểm dữ liệu nằm về một phía của siêu phẳng, nó sẽ được gán vào lớp thứ nhất; nếu nằm về phía còn lại, nó sẽ được gán vào lớp thứ hai. Nhờ cơ chế tối đa hóa biên phân tách, SVM thường đạt hiệu quả cao trong các bài toán phân loại văn bản nhị phân như phát hiện tin giả.



Hình 1.2: Minh họa thuật toán nhị phân SVM (phỏng theo [2])

1.3.3 Biểu diễn đặc trưng cơ bản

1.3.3.1 Tần suất từ – Tần suất tài liệu nghịch đảo (TF-IDF)

TF-IDF (Term Frequency – Inverse Document Frequency) [10] là một phương pháp biểu diễn văn bản được sử dụng phổ biến trong các bài toán xử lý ngôn ngữ tự nhiên và phân loại văn bản. Phương pháp này nhằm đánh giá mức độ quan trọng của một từ trong một tài liệu dựa trên tần suất xuất hiện của từ đó trong tài liệu và mức độ phổ biến của nó trong toàn bộ tập dữ liệu.

Term Frequency (TF) biểu diễn số lần một từ t xuất hiện trong tài liệu d . Để tránh thiên lệch do độ dài tài liệu, TF thường được chuẩn hóa theo tổng số từ trong tài liệu:

$$TF(t, d) = \frac{f_{t,d}}{\sum_{v' \in d} f_{v',d}}$$

trong đó $f_{t,d}$ là số lần xuất hiện của từ t trong tài liệu d .

Inverse Document Frequency (IDF) được sử dụng nhằm giảm trọng số của các từ xuất hiện phổ biến trong nhiều tài liệu và tăng trọng số cho các từ mang tính phân biệt cao. IDF được tính như sau:

$$IDF(t) = \log \left(\frac{N}{df_t} \right)$$

trong đó N là tổng số tài liệu trong tập dữ liệu và df_t là số tài liệu chứa từ t .

Giá trị TF-IDF của một từ t trong tài liệu d được xác định bằng tích của hai thành phần trên:

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

Kết quả thu được là một trọng số thực phản ánh mức độ quan trọng tương đối của từ trong tài liệu. Sau khi tính toán cho toàn bộ từ vựng, mỗi tài liệu được biểu diễn dưới dạng một vector đặc trưng có kích thước bằng số lượng từ trong từ điển. Các vector này thường có tính thưa (sparse) và được sử dụng làm đầu vào cho các mô hình học máy truyền thống như Hồi quy Logistic (Logistic Regression) và Máy vectơ hỗ trợ (Support Vector Machine) trong các bài toán phân loại văn bản, bao gồm phát hiện tin giả.

1.4 Học sâu

1.4.1 Định nghĩa

Học sâu (Deep Learning) là một nhánh của học máy dựa trên các mô hình mạng nơ-ron nhân tạo có nhiều lớp ẩn, cho phép hệ thống học được các đặc trưng phức tạp từ dữ liệu đầu vào. Khác với các phương pháp học máy truyền thống, trong đó đặc trưng thường được thiết kế thủ công, học sâu có khả năng tự động trích xuất và biểu diễn đặc trưng thông qua quá trình huấn luyện trên tập dữ liệu lớn.

Mỗi mô hình học sâu được cấu thành từ nhiều lớp nơ-ron liên kết với nhau theo từng tầng. Dữ liệu đầu vào sẽ được truyền qua các lớp này theo cơ chế lan truyền tiến (feedforward), tại đó mỗi lớp sẽ thực hiện các phép biến đổi tuyến tính kết hợp với hàm kích hoạt phi tuyến nhằm tạo ra biểu diễn ở mức trừu tượng cao hơn. Trong quá trình huấn luyện, các tham số của mô hình được điều chỉnh thông qua thuật toán lan truyền ngược (backpropagation) nhằm tối thiểu hóa hàm mất mát.

Nhờ cấu trúc nhiều tầng và khả năng học biểu diễn tự động, học sâu đã đạt được hiệu quả cao trong nhiều lĩnh vực như thị giác máy tính, xử lý ngôn ngữ tự nhiên

và nhận dạng tiếng nói. Trong bài toán phát hiện tin giả, các mô hình học sâu như BiLSTM và PhoBERT cho phép khai thác thông tin ngữ cảnh và quan hệ giữa các từ trong văn bản một cách hiệu quả hơn so với các phương pháp truyền thống.

1.4.2 Mạng nơ-ron nhân tạo (Artificial Neural Network)

Dựa trên cơ chế hoạt động của các neuron trong hệ thần kinh sinh học, các nhà nghiên cứu đã xây dựng mô hình mạng nơ-ron nhân tạo nhằm mô phỏng quá trình xử lý thông tin của não người. Trong cơ thể, neuron được xem là đơn vị cấu trúc cơ bản của hệ thần kinh. Mỗi neuron bao gồm thân tế bào chứa nhân, các sợi nhánh (dendrite) có chức năng tiếp nhận tín hiệu từ các neuron khác và sợi trục (axon) dùng để truyền tín hiệu đi.

Khi một neuron nhận được các tín hiệu đầu vào thông qua các sợi nhánh, nó sẽ tiến hành xử lý và tổng hợp các tín hiệu này. Nếu mức tín hiệu sau khi xử lý vượt qua một giá trị ngưỡng nhất định, neuron sẽ phát sinh tín hiệu đầu ra và truyền qua sợi trục đến các neuron kế tiếp. Cơ chế này được trừu tượng hóa trong mạng nơ-ron nhân tạo dưới dạng một hàm toán học, trong đó nhiều giá trị đầu vào được kết hợp để tạo ra một đầu ra duy nhất. Vai trò của ngưỡng kích hoạt được mô phỏng thông qua hàm kích hoạt (Activation Function), quyết định xem neuron có phát tín hiệu hay không.

Mạng neuron nhân tạo là một tập hợp gồm các neuron nhân tạo được sắp xếp theo từng lớp ẩn. Mỗi lớp ẩn (hidden layer) bao gồm một hoặc nhiều neuron nhân tạo. Các neuron này liên kết từng đôi một với các neuron ở lớp kế tiếp và sẽ không có sự liên kết giữa các neuron trong cùng một lớp. Số lớp của một mạng neuron nhân tạo được tính bằng tổng số lớp ẩn và lớp đầu vào, lớp đầu ra không được tính. Trong quá trình huấn luyện, mạng nơ-ron nhân tạo thực hiện cơ chế lan truyền tiến (feedforward), trong đó dữ liệu đầu vào được truyền qua từng lớp và được biến đổi thông qua các trọng số tương ứng để tạo ra kết quả dự đoán ở đầu ra. Để đánh giá mức độ chính xác của dự đoán này, một hàm mất mát (Loss Function) được sử dụng nhằm đo lường sai số giữa giá trị dự đoán và nhãn thực tế. Dựa trên giá trị sai số đó, các tham số của mô hình sẽ được điều chỉnh thông qua thuật toán lan truyền ngược nhằm giảm thiểu sai số. Quá trình này được lặp lại nhiều lần cho đến khi mô hình đạt được mức sai số nhỏ nhất có thể.

Trong các kiến trúc mạng nơ-ron hiện đại, nhiều mô hình đã được phát triển nhằm xử lý các loại dữ liệu khác nhau. Đối với bài toán xử lý văn bản, các kiến trúc mạng nơ-ron hồi quy (Recurrent Neural Network – RNN) và các mô hình dựa trên Transformer được sử dụng phổ biến.

1.4.2.1 Một số thành phần cần chú ý

Hàm kích hoạt (Activation Function) là thành phần quan trọng trong mạng nơ-ron nhân tạo, giữ vai trò quyết định xem tín hiệu sau khi được tổng hợp tại một neuron có được truyền tiếp đến lớp kế tiếp hay không. Sau khi các giá trị đầu vào được nhân với trọng số và cộng lại, kết quả thu được sẽ được đưa qua một hàm kích hoạt nhằm tạo ra tính phi tuyến cho mô hình. Nhờ tính phi tuyến này, mạng nơ-ron có khả năng học được các mối quan hệ phức tạp trong dữ liệu. Một số hàm kích hoạt thường được sử dụng gồm Sigmoid, Tanh và ReLU. Đối với các bài toán phân loại, đặc biệt là phân loại nhị phân, hàm Sigmoid thường được dùng ở lớp đầu ra để đưa kết quả dự đoán về dạng xác suất trong khoảng từ 0 đến 1, trong khi hàm Softmax được sử dụng khi cần phân phối xác suất trên nhiều lớp.

Để đánh giá mức độ chính xác của mô hình trong quá trình huấn luyện, người ta sử dụng hàm mất mát (Loss Function) nhằm đo lường sai số giữa giá trị dự đoán và nhãn thực tế. Trong các bài toán phân loại văn bản, hàm mất mát phổ biến là Cross-Entropy, do khả năng phản ánh tốt sự khác biệt giữa phân phối xác suất dự đoán và phân phối thực tế. Giá trị của hàm mất mát càng nhỏ thì mô hình càng đưa ra dự đoán gần với nhãn đúng.

Quá trình huấn luyện mạng nơ-ron nhân tạo diễn ra thông qua hai bước chính là lan truyền tiến (feedforward) và lan truyền ngược (backpropagation). Ở bước lan truyền tiến, dữ liệu đầu vào được truyền qua từng lớp của mạng, tại đó các phép biến đổi tuyến tính và phi tuyến được thực hiện để tạo ra kết quả dự đoán ở đầu ra. Sau khi tính được giá trị của hàm mất mát, thuật toán lan truyền ngược sẽ được áp dụng để tính toán đạo hàm của sai số theo từng tham số trong mạng. Các tham số này sau đó được cập nhật bằng thuật toán tối ưu hóa, phổ biến nhất là Gradient Descent, nhằm giảm dần giá trị sai số qua từng vòng lặp huấn luyện.

Trong quá trình huấn luyện các mạng nhiều tầng, có thể xuất hiện các hiện tượng như mất mát hoặc bùng nổ gradient khi giá trị đạo hàm trở nên quá nhỏ hoặc quá lớn. Những vấn đề này ảnh hưởng đến tốc độ và độ ổn định của quá trình học. Các kiến trúc hiện đại như LSTM và Transformer được thiết kế nhằm hạn chế những hạn chế này, giúp mô hình học được các quan hệ dài hạn trong dữ liệu chuỗi, đặc biệt hiệu quả trong các bài toán xử lý ngôn ngữ tự nhiên như phát hiện tin giả.

1.4.3 Mạng nơ-ron hồi quy (Recurrent Neural Network – RNN)

RNN là một mô hình học sâu được thiết kế để xử lý dữ liệu tuần tự bằng cách duy trì thông tin từ các trạng thái trước đó. Điều này giúp RNN có khả năng ghi nhớ và sử dụng các mối liên hệ giữa các phần tử trong chuỗi dữ liệu, tạo ra đầu ra có ngữ cảnh

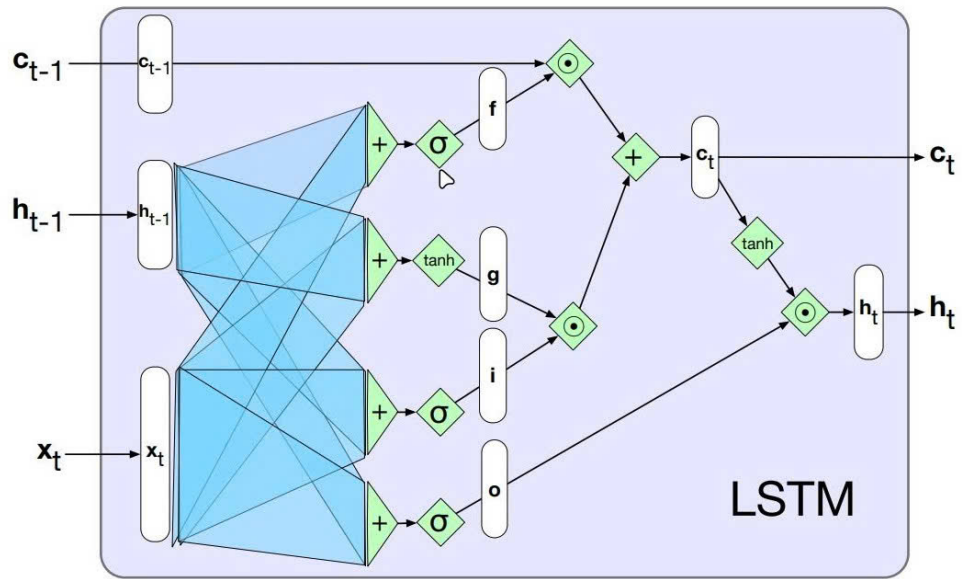
và ý nghĩa hơn.

Dữ liệu tuần tự có thể bao gồm văn bản, chuỗi thời gian, hoặc tín hiệu âm thanh, trong đó các thành phần có sự liên kết với nhau dựa trên ngữ nghĩa hoặc quy luật cụ thể. Khác với các mạng nơ-ron truyền thống xử lý dữ liệu đầu vào độc lập, RNN có một vòng lặp bên trong, cho phép lưu trữ trạng thái và truyền tải thông tin qua từng bước thời gian.

Tuy nhiên, mặc dù có khả năng ghi nhớ thông tin qua các bước thời gian, RNN truyền thống vẫn tồn tại một số hạn chế. Khi chuỗi dữ liệu có độ dài lớn, quá trình lan truyền ngược sai số qua nhiều bước thời gian có thể làm cho gradient trở nên rất nhỏ hoặc rất lớn. Hiện tượng này được gọi là mất mát gradient (vanishing gradient) hoặc bùng nổ gradient (exploding gradient). Khi gradient bị suy giảm quá mức, mô hình sẽ gặp khó khăn trong việc học và ghi nhớ các phụ thuộc dài hạn giữa các phần tử trong chuỗi. Điều này ảnh hưởng trực tiếp đến hiệu quả của RNN trong các bài toán xử lý văn bản, nơi mà ngữ nghĩa của một từ có thể phụ thuộc vào các từ xuất hiện ở vị trí xa trong câu hoặc đoạn văn. Để khắc phục những hạn chế này, các kiến trúc cải tiến như Bộ nhớ dài-ngắn hạn (Long Short-Term Memory – LSTM) đã được đề xuất nhằm tăng cường khả năng ghi nhớ thông tin dài hạn và cải thiện hiệu suất của mô hình trên dữ liệu tuần tự.

1.4.4 Bộ nhớ dài-ngắn hạn (Long Short-Term Memory – LSTM)

Bộ nhớ dài-ngắn hạn (Long Short-Term Memory – LSTM) [11] là một kiến trúc RNN được thiết kế nhằm khắc phục hạn chế trong việc ghi nhớ thông tin dài hạn. Trong khi RNN chỉ sử dụng một trạng thái ẩn đơn giản để truyền thông tin qua các bước thời gian, thì LSTM bổ sung một bộ nhớ tế bào (memory cell) cùng với ba cơ chế cổng kiểm soát: cổng quên, cổng đầu vào và cổng đầu ra. Các cổng này cho phép mô hình lựa chọn thông tin cần ghi nhớ hoặc loại bỏ, nhờ đó duy trì được ngữ cảnh trong các chuỗi dữ liệu dài.



Hình 1.3: Minh họa cấu trúc mạng LSTM [3].

Điểm nổi bật của LSTM chính là khả năng học và lưu giữ các phụ thuộc dài hạn (long-term dependencies) trong dữ liệu tuần tự. Điều này đặc biệt hữu ích trong các ứng dụng yêu cầu hiểu ngữ cảnh kéo dài như xử lý ngôn ngữ tự nhiên (NLP), dịch máy, nhận dạng giọng nói, hay dự báo chuỗi thời gian.

Bằng cách giải quyết bài toán gradient biến mất - vốn là rào cản lớn trong việc huấn luyện các mạng hồi tiếp sâu, LSTM không chỉ đảm bảo quá trình học ổn định mà còn đặc biệt hiệu quả trong các bài toán xử lý văn bản như phân loại tin tức.

Trong cấu trúc của LSTM, mỗi cell được điều khiển bởi ba cổng chính nhằm kiểm soát luồng thông tin. Cổng quên (forget gate) có nhiệm vụ xác định những thông tin nào từ trạng thái trước đó cần được giữ lại hoặc loại bỏ. Cổng này giúp mô hình loại bỏ các thông tin không còn quan trọng, từ đó tránh nhiễu trong quá trình ghi nhớ dài hạn. Tiếp theo, cổng vào (input gate) quyết định những thông tin mới nào từ dữ liệu hiện tại sẽ được cập nhật vào bộ nhớ cell. Sự kết hợp giữa cổng quên và cổng vào cho phép LSTM duy trì và cập nhật trạng thái bộ nhớ một cách có chọn lọc. Cuối cùng, cổng ra (output gate) xác định phần thông tin nào từ bộ nhớ sẽ được sử dụng để tạo ra trạng thái ẩn tại thời điểm hiện tại, từ đó ảnh hưởng đến đầu ra của mô hình. Nhờ cơ chế điều khiển thông tin thông qua các cổng này, LSTM có khả năng học và ghi nhớ các phụ thuộc dài hạn trong chuỗi dữ liệu hiệu quả hơn so với RNN truyền thống.

Tuy nhiên, LSTM tiêu chuẩn chỉ khai thác thông tin theo một chiều thời gian, tức là từ quá khứ đến hiện tại. Trong các bài toán xử lý ngôn ngữ tự nhiên, ngữ nghĩa của một từ không chỉ phụ thuộc vào các từ đứng trước mà còn có thể chịu ảnh hưởng bởi các từ xuất hiện phía sau trong câu. Do đó, để tận dụng thông tin ngữ cảnh theo cả hai hướng, kiến trúc Mạng LSTM hai chiều (Bidirectional LSTM – BiLSTM) đã được

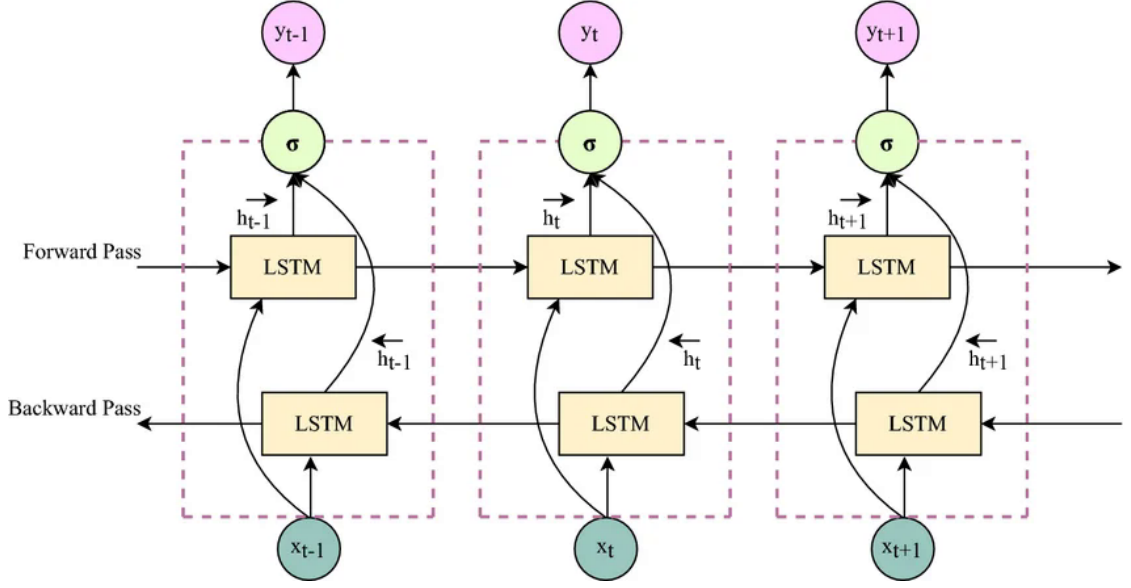
đề xuất nhằm mở rộng khả năng biểu diễn của LSTM trong các bài toán phân loại văn bản như phát hiện tin giả.

1.4.5 Mạng LSTM hai chiều (Bidirectional LSTM)

Mạng LSTM hai chiều (Bidirectional LSTM – BiLSTM) là một kiến trúc mở rộng của LSTM nhằm khai thác thông tin ngữ cảnh theo cả hai hướng của chuỗi dữ liệu. Trong khi LSTM truyền thống chỉ xử lý chuỗi theo một chiều thời gian, thường là từ trái sang phải (từ quá khứ đến hiện tại), thì BiLSTM bao gồm hai mạng LSTM hoạt động song song. Một mạng xử lý chuỗi theo chiều thuận (forward), tức là từ phần tử đầu tiên đến phần tử cuối cùng; mạng còn lại xử lý chuỗi theo chiều ngược (backward), tức là từ cuối về đầu.

Cụ thể, với mỗi bước thời gian, mạng LSTM thuận sẽ tạo ra một trạng thái ẩn dựa trên các phần tử đã xuất hiện trước đó trong chuỗi. Đồng thời, mạng LSTM ngược sẽ tạo ra một trạng thái ẩn khác dựa trên các phần tử xuất hiện phía sau. Hai trạng thái ẩn này sau đó được kết hợp lại, thường thông qua phép nối (concatenation), để tạo thành biểu diễn cuối cùng tại thời điểm đó. Nhờ cơ chế này, mô hình có thể đồng thời tận dụng thông tin từ cả ngữ cảnh trước và sau của một từ trong câu.

Trong các bài toán xử lý ngôn ngữ tự nhiên, ý nghĩa của một từ không chỉ phụ thuộc vào các từ đứng trước mà còn chịu ảnh hưởng bởi các từ xuất hiện phía sau. Do đó, BiLSTM cho phép mô hình nắm bắt được mối quan hệ ngữ cảnh toàn diện hơn so với LSTM một chiều. Trong bài toán phát hiện tin giả, việc khai thác đầy đủ ngữ cảnh của câu và đoạn văn giúp mô hình hiểu rõ hơn nội dung và sắc thái của thông tin, từ đó nâng cao hiệu quả phân loại giữa tin “thật” và tin “giả”.



Hình 1.4: Minh họa cấu trúc mạng LSTM hai chiều [4].

1.4.6 Cơ chế chú ý (Attention Mechanism)

Cơ chế chú ý (Attention Mechanism) [12] được đề xuất nhằm khắc phục hạn chế của các mô hình xử lý chuỗi khi phải nén toàn bộ thông tin của một câu vào một vector duy nhất. Thay vì xem mọi phần tử trong chuỗi có mức độ quan trọng như nhau, cơ chế chú ý cho phép mô hình tự động học trọng số cho từng bước thời gian, từ đó tập trung nhiều hơn vào những thành phần mang thông tin quan trọng đối với nhiệm vụ dự đoán.

Giả sử $\{h_1, h_2, \dots, h_T\}$ là các trạng thái ẩn thu được từ mô hình LSTM hoặc BiLSTM. Tại mỗi bước thời gian t , một điểm số e_t được tính để đo mức độ liên quan của trạng thái ẩn h_t đối với đầu ra cuối cùng:

$$e_t = \mathbf{v}^\top \tanh(\mathbf{W}_a \overleftrightarrow{\mathbf{h}}_t) \quad (1.1)$$

Các điểm số này sau đó được chuẩn hóa thông qua hàm softmax để tạo thành trọng số chú ý α_t :

$$\alpha_t = \frac{\exp(e_t)}{\sum_{k=1}^T \exp(e_k)} \quad (1.2)$$

Cuối cùng, vector ngữ cảnh $\mathbf{c}$ được tính bằng tổng có trọng số của các trạng thái ẩn:

$$\mathbf{c} = \sum_{t=1}^T \alpha_t \overleftrightarrow{\mathbf{h}}_t \quad (1.3)$$

Trong đó, α_t phản ánh mức độ đóng góp của từng từ hoặc từng bước thời gian vào biểu diễn cuối cùng của câu. Những từ quan trọng sẽ nhận được trọng số lớn hơn, trong khi các từ ít liên quan sẽ có trọng số nhỏ hơn. Nhờ đó, mô hình có khả năng tập trung vào các thành phần mang tính quyết định trong văn bản thay vì xử lý toàn bộ chuỗi một cách đồng đều.

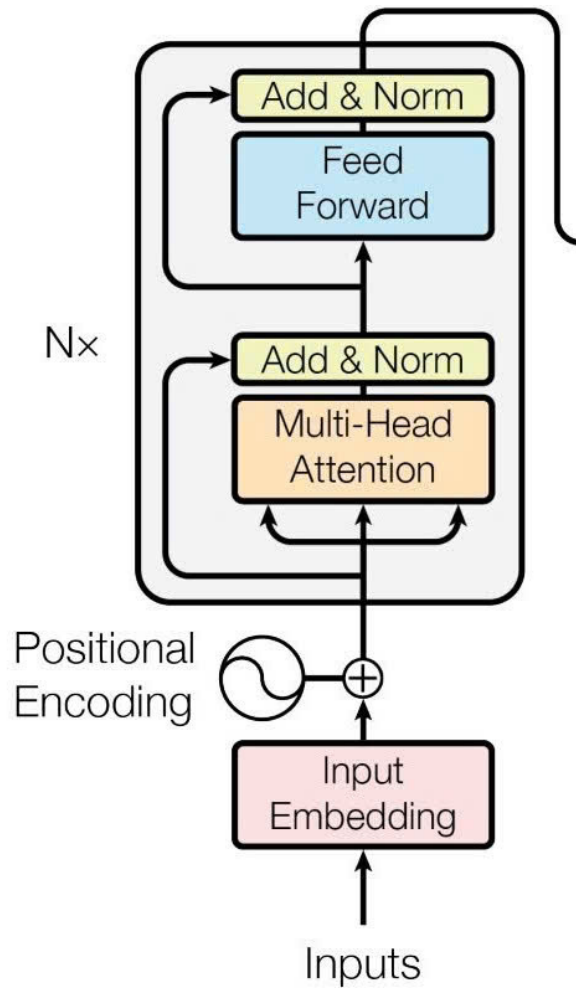
Trong bài toán phát hiện tin giả, cơ chế chú ý giúp mô hình xác định những từ hoặc cụm từ có ảnh hưởng lớn đến nội dung và ý nghĩa của tin tức, chẳng hạn như các từ thể hiện quan điểm cực đoan, thông tin gây hiểu lầm hoặc các chi tiết bất thường. Điều này góp phần nâng cao khả năng phân biệt giữa tin “thật” và tin “giả”.

Đáng chú ý, cơ chế tự chú ý (self-attention) là nền tảng cốt lõi của kiến trúc Transformer. Thay vì chỉ tính trọng số dựa trên một trạng thái ẩn duy nhất, self-attention cho phép mỗi phần tử trong chuỗi tương tác trực tiếp với tất cả các phần tử khác, từ đó xây dựng biểu diễn ngữ cảnh toàn cục hiệu quả hơn. Đây chính là cơ sở cho các mô hình ngôn ngữ hiện đại như BERT và PhoBERT được trình bày ở phần tiếp theo.

1.4.7 Mô hình Transformer và PhoBERT

Transformer là một kiến trúc học sâu dựa trên cơ chế tự chú ý (self-attention) [5], được đề xuất nhằm khắc phục những hạn chế của các mô hình tuần tự như RNN và LSTM trong việc xử lý phụ thuộc dài hạn. Không giống như các mô hình xử lý dữ liệu theo từng bước thời gian, Transformer cho phép xử lý toàn bộ chuỗi đầu vào đồng thời, từ đó tăng khả năng song song hóa và cải thiện hiệu quả huấn luyện.

Kiến trúc Transformer ban đầu bao gồm hai thành phần chính là encoder và decoder. Tuy nhiên, trong các mô hình ngôn ngữ tiền huấn luyện như BERT [13] và PhoBERT [14], chỉ phần encoder được sử dụng để xây dựng biểu diễn ngữ cảnh cho văn bản. Do đó, trong phạm vi đề tài này, trọng tâm được đặt vào kiến trúc Transformer encoder.



Hình 1.5: Minh họa khối encoder của Transformer [5].

Hình trên minh họa cấu trúc của một khối encoder trong Transformer. Đầu vào của mô hình là các vector embedding của từ kết hợp với thông tin vị trí (positional encoding) nhằm bảo toàn thứ tự của các từ trong câu. Vì cơ chế self-attention không xử lý dữ liệu theo thứ tự tuần tự, positional encoding đóng vai trò cung cấp thông tin về vị trí tương đối của các từ trong chuỗi.

Mỗi khối encoder bao gồm hai thành phần chính. Thành phần thứ nhất là cơ chế multi-head self-attention, cho phép mỗi từ trong câu tương tác trực tiếp với tất cả các từ khác để tính toán mức độ liên quan. Nhờ đó, mô hình có thể nắm bắt được các quan hệ ngữ nghĩa dài hạn trong văn bản mà không bị giới hạn bởi khoảng cách vị trí. Kết quả của lớp attention được đưa qua bước cộng và chuẩn hóa (Add & Norm) nhằm ổn định quá trình huấn luyện.

Thành phần thứ hai là mạng truyền thẳng theo từng vị trí (Positionwise Feed-Forward Network – FFN), thực hiện các phép biến đổi phi tuyến trên từng vector đầu ra của lớp attention. Sau đó, một bước Add & Norm tiếp theo được áp dụng. Các khối

encoder này được xếp chồng nhiều lần nhằm tăng khả năng biểu diễn của mô hình.

Nhờ khả năng học biểu diễn ngữ cảnh toàn cục thông qua self-attention và xử lý song song hiệu quả, Transformer đã trở thành nền tảng cho nhiều mô hình ngôn ngữ hiện đại. PhoBERT [14] được xây dựng dựa trên kiến trúc RoBERTa [15], sử dụng Transformer encoder nhiều tầng và được tiền huấn luyện trên lượng lớn dữ liệu tiếng Việt.

Trong bài toán phát hiện tin giả, vector tương ứng với token đặc biệt [CLS] được sử dụng làm biểu diễn toàn cục của văn bản và được đưa qua một lớp phân loại để dự đoán nhãn “thật” hoặc “giả”. Việc tận dụng biểu diễn ngữ cảnh sâu từ PhoBERT giúp mô hình hiểu rõ hơn nội dung và sắc thái của văn bản, từ đó nâng cao hiệu quả phân loại.

1.5 Kết luận chương

Chương này đã trình bày nền tảng lý thuyết cho bốn mô hình so sánh trong nghiên cứu. Từ LR tuyến tính đến SVM phi tuyến, từ BiLSTM tuần tự đến Transformer song song, mỗi mô hình phản ánh một giai đoạn phát triển của NLP. Chương tiếp theo mô tả bộ dữ liệu và quy trình tiền xử lý.

Chương 2 Bộ dữ liệu và tiền xử lý

2.1 Thu thập và mô tả bộ dữ liệu

2.1.1 Nguồn dữ liệu

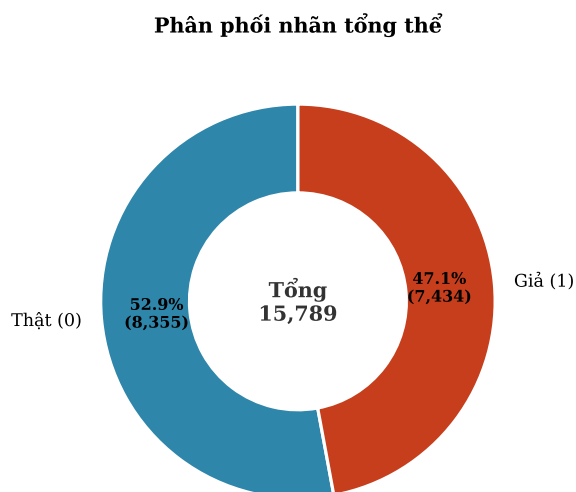
Bộ dữ liệu bao gồm các bài viết tiếng Việt được thu thập từ hai nguồn chính: các bài đăng công khai trên Facebook và một số bài báo cũ trong thời gian diễn ra COVID-19. Mỗi bản ghi chứa tối thiểu hai trường chính:

- **text**: nội dung văn bản của bài viết.
- **label**: nhãn phân loại (0 = Thật, 1 = Giả).

Sau khi thực hiện bước phân đoạn từ (*word segmentation*), dữ liệu được lưu lại và được sử dụng làm đầu vào cho quá trình làm sạch và chia tập.

Tổng số bản ghi ban đầu của bộ dữ liệu là **15,789 mẫu**.

2.1.2 Phân phối nhãn



Hình 2.1: Phân phối nhãn theo từng tập dữ liệu và tổng quan toàn bộ 15,789 mẫu

Hình 2.1 minh họa phân phối nhãn tổng thể của toàn bộ tập dữ liệu. Có thể thấy, tỷ lệ tin thật và tin giả khá cân bằng, giúp đảm bảo mô hình không bị thiên lệch về một phía trong quá trình huấn luyện và đánh giá. Sự cân bằng này là yếu tố quan trọng để kết quả thực nghiệm phản ánh đúng hiệu quả của các phương pháp phân loại, đặc biệt trong bối cảnh dữ liệu thực tế thường có sự chênh lệch giữa các lớp.

2.2 Tiền xử lý văn bản

2.2.1 Làm sạch và lọc dữ liệu

Trước khi thực hiện bước chia tập huấn luyện, dữ liệu được tiến hành làm sạch nhằm loại bỏ nhiễu và hạn chế nguy cơ rò rỉ thông tin. Quy trình làm sạch bao gồm các bước sau:

1. **Loại bỏ văn bản trùng lặp:** Các bản ghi có nội dung `text` giống hệt nhau được loại bỏ, chỉ giữ lại lần xuất hiện đầu tiên.
2. **Loại bỏ văn bản rỗng hoặc giá trị thiếu:** Các bản ghi có trường `text` bị thiếu (NaN) hoặc chuỗi rỗng sau khi loại bỏ khoảng trắng được loại khỏi tập dữ liệu.
3. **Loại bỏ văn bản quá ngắn:** Những văn bản có ít hơn 10 từ bị loại bỏ do không cung cấp đủ thông tin ngữ nghĩa cho bài toán phân loại. Ngưỡng này được cấu hình thông qua tham số `min_word_count = 10`.
4. **Đặt lại chỉ mục (index reset):** Sau khi hoàn tất các bước trên, dữ liệu được đánh lại chỉ mục nhằm đảm bảo tính nhất quán và thuận tiện cho quá trình xử lý tiếp theo.

Sau quá trình làm sạch:

- Số mẫu ban đầu: 15,789
- Số mẫu bị loại bỏ: 1,831
- Số mẫu còn lại: 13,958

Quy trình làm sạch này giúp giảm nhiễu dữ liệu và đảm bảo rằng quá trình huấn luyện cũng như đánh giá mô hình không bị ảnh hưởng bởi các bản ghi trùng lặp hoặc không hợp lệ.

2.2.2 Phân đoạn từ bằng VnCoreNLP

Tiếng Việt là một ngôn ngữ đơn lập, trong đó các âm tiết được phân tách bằng khoảng trắng; tuy nhiên, ranh giới từ không hoàn toàn trùng khớp với ranh giới âm tiết. Chẳng hạn, cụm từ “Việt Nam” bao gồm hai âm tiết tách biệt nhưng được xem như một đơn vị từ vựng duy nhất. Đặc điểm này đặt ra thách thức đáng kể cho các hệ thống xử lý ngôn ngữ tự nhiên, bởi việc coi mỗi âm tiết là một từ độc lập có thể làm sai lệch cấu trúc ngữ nghĩa của văn bản. Do đó, phân đoạn từ là bước tiền xử lý cần thiết nhằm đảm bảo tính chính xác trong quá trình biểu diễn và trích xuất đặc trưng.

Trong nghiên cứu này, quá trình phân đoạn từ được thực hiện bởi thư viện `py_vncorenlp`, sử dụng bộ tách từ `RDRSegmenter` của VnCoreNLP [6]. Đây cũng là công cụ phân đoạn được sử dụng trong giai đoạn tiền huấn luyện của mô hình PhoBERT. Việc áp dụng cùng một bộ phân đoạn giúp duy trì tính nhất quán giữa dữ liệu đầu vào và từ vựng của mô hình ngôn ngữ tiền huấn luyện, từ đó giảm thiểu sai lệch trong không gian biểu diễn và nâng cao khả năng tổng quát hóa của mô hình.

Quy trình phân đoạn được triển khai theo các bước sau:

- Tự động tải và lưu trữ mô hình VnCoreNLP tại thư mục cục bộ (`.vncorenlp`) nếu chưa tồn tại.
- Thực hiện phân đoạn đối với cả hai trường `title` và `text`.
- Chuẩn hóa văn bản trước khi phân đoạn, bao gồm:
 - Loại bỏ dấu gạch dưới phát sinh từ các lần phân đoạn trước (nếu có).
 - Chuẩn hóa khoảng trắng nhằm đảm bảo tính đồng nhất định dạng.

Sau khi hoàn tất quá trình phân đoạn, các từ đa âm tiết được nối với nhau bằng dấu gạch dưới. Ví dụ:

Trước: “Chính phủ Việt Nam công bố chính sách mới”

Sau: “Chính_phủ Việt_Nam công_bố chính_sách mới”

Bước phân đoạn từ đóng vai trò then chốt trong việc cải thiện chất lượng biểu diễn đầu vào, đặc biệt đối với các mô hình dựa trên cơ chế *subword tokenization* như PhoBERT. Nhờ đảm bảo sự tương thích giữa dữ liệu đã xử lý và từ điển tiền huấn luyện, quá trình này góp phần nâng cao độ ổn định và hiệu quả của mô hình trong giai đoạn huấn luyện và suy diễn.

2.3 Chia tập dữ liệu

2.3.1 Chia phân tầng

Sau khi thực hiện phân đoạn từ, dữ liệu được chia thành ba tập: huấn luyện (train), kiểm định (validation) và kiểm tra (test). Quy trình này bao gồm bước làm sạch dữ liệu trước khi chia tập.

Việc chia tập được thực hiện theo chiến lược phân tầng (*stratified split*) nhằm duy trì phân phối nhãn đồng đều trong cả ba tập. Cấu hình chia tập như sau:

- Train: 70%
- Validation: 15%
- Test: 15%
- `random_state = 42`

Quy trình chia được thực hiện theo hai bước. Đầu tiên, 15% dữ liệu được tách ra làm tập kiểm tra (test set). Sau đó, từ 85% dữ liệu còn lại, tiếp tục tách một phần tương ứng 15% tổng dữ liệu ban đầu (xấp xỉ 12.75% của phần còn lại) để tạo tập kiểm định (validation set).

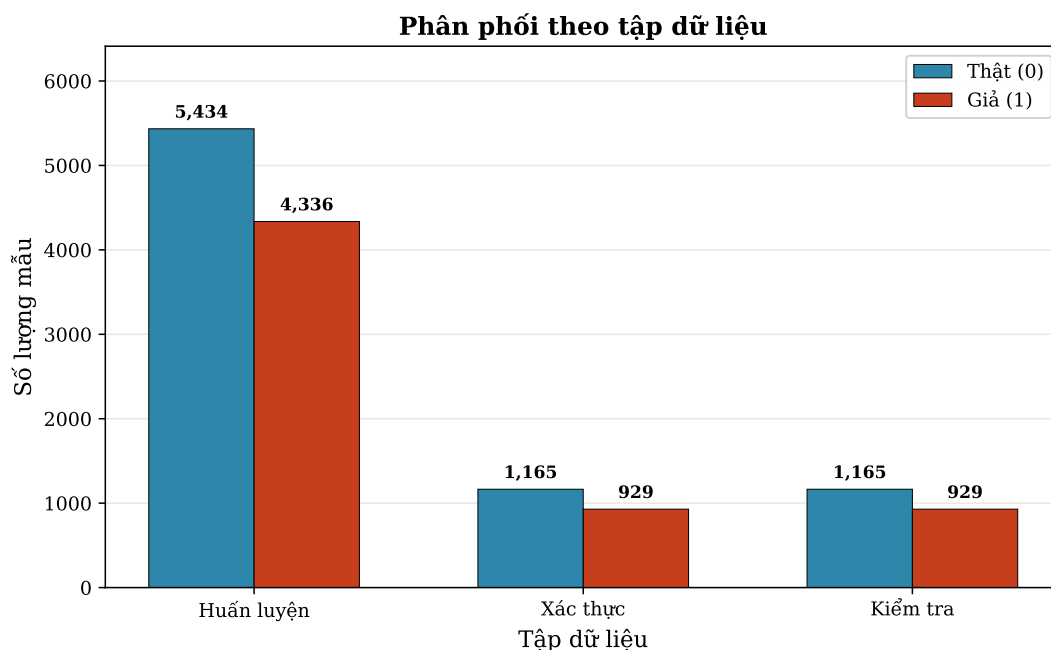
Tham số `stratify = label` đảm bảo rằng phân phối nhãn được bảo toàn giữa các tập:

$$P_{\text{train}}(y) \approx P_{\text{val}}(y) \approx P_{\text{test}}(y)$$

Nhờ đó, sai lệch phân phối nhãn giữa các tập được hạn chế, giúp quá trình đánh giá mô hình phản ánh chính xác hơn khả năng tổng quát hóa.

Kết quả chia tập cuối cùng:

- Train: 9,770 mẫu
- Validation: 2,094 mẫu
- Test: 2,094 mẫu



Hình 2.2: Phân phối nhãn trong các tập dữ liệu huấn luyện, kiểm định và kiểm tra

Bảng 2.1 mô tả thống kê chi tiết sau khi chia tập:

Bảng 2.1: Thống kê tập dữ liệu

Tập	Tổng	Tin thật	Tin giả	Độ dài TB
Huấn luyện	9,770	5,434(55.6%)	4,336(44.4%)	90.1
Xác thực	2,094	1,165(55.6%)	929(44.4%)	91.7
Kiểm tra	2,094	1,165(55.6%)	929(44.4%)	96.5

Bảng 2.1 trình bày số lượng mẫu, phân phối nhãn và độ dài trung bình của văn bản trong từng tập dữ liệu. Có thể thấy rằng tin thật chiếm 55.6% và tin giả chiếm 44.4% ở cả ba tập (huấn luyện, kiểm định và kiểm tra), cho thấy chiến lược chia dữ liệu theo phương pháp phân tầng (*stratified split*) đã bảo toàn phân phối nhãn ban đầu.

Độ dài trung bình của văn bản trong tập kiểm định và kiểm tra (91.7 từ đối với tập kiểm định và 96.5 từ đối với tập kiểm tra) cao hơn nhẹ so với tập huấn luyện (90.1 từ), phản ánh sự đa dạng về cấu trúc văn bản trong toàn bộ dữ liệu. Mặc dù tồn tại sự mất cân bằng lớp ở mức vừa phải, tỷ lệ này không quá nghiêm trọng và được xử lý thông qua cơ chế trọng số lớp (*class weighting*) trong quá trình huấn luyện nhằm giảm thiểu sai lệch dự đoán.

2.3.2 Kiểm tra rò rỉ dữ liệu

Để đảm bảo tính khách quan và độ tin cậy của quá trình đánh giá, nhóm tiến hành kiểm tra hiện tượng rò rỉ dữ liệu (*data leakage*) giữa các tập huấn luyện, kiểm định và kiểm tra sau khi thực hiện chia dữ liệu.

Cụ thể, toàn bộ nội dung văn bản trong từng tập (Train, Validation, Test) được chuyển đổi thành các tập hợp (*set*) dựa trên giá trị của trường `text`. Sau đó, nhóm thực hiện phép giao giữa từng cặp tập dữ liệu nhằm phát hiện các bản ghi trùng lặp:

- Train và Validation
- Train và Test
- Validation và Test

Kết quả cho thấy:

$$\text{Train} \cap \text{Validation} = \emptyset$$

$$\text{Train} \cap \text{Test} = \emptyset$$

$$\text{Validation} \cap \text{Test} = \emptyset$$

Không phát hiện văn bản trùng lặp giữa bất kỳ cặp tập dữ liệu nào.

Việc kiểm tra rò rỉ dữ liệu đặc biệt quan trọng trong các bài toán phân loại văn bản, bởi sự xuất hiện của các mẫu trùng lặp giữa tập huấn luyện và tập kiểm tra có thể dẫn đến đánh giá thiên lệch (*biased evaluation*). Khi đó, mô hình có thể đạt hiệu năng cao một cách giả tạo do đã “ghi nhớ” nội dung từ tập huấn luyện, thay vì thực sự học được các đặc trưng có khả năng tổng quát hóa. Do đó, việc đảm bảo tính độc lập giữa các tập dữ liệu là điều kiện tiên quyết để đánh giá chính xác năng lực của mô hình.

2.4 Kết luận chương

Chương này đã trình bày toàn bộ quy trình chuẩn bị dữ liệu cho bài toán phát hiện tin giả tiếng Việt, bao gồm phân đoạn từ, làm sạch và chia tập theo chiến lược phân tầng. Dữ liệu được phân đoạn bằng RDRSegmenter nhằm đảm bảo tính tương thích với mô hình PhoBERT, sau đó được loại bỏ các văn bản trùng lặp, rỗng hoặc quá ngắn nhằm giảm nhiễu và nâng cao chất lượng đầu vào.

Tập dữ liệu sau làm sạch được chia theo tỷ lệ 70%–15%–15% cho các tập huấn luyện, kiểm định và kiểm tra với `random_state = 42`. Đồng thời, hệ thống tiến hành

kiểm tra nghiêm ngặt rõ ràng dữ liệu giữa các tập để đảm bảo không tồn tại giao nhau về nội dung văn bản.

Quy trình này đảm bảo tính nhất quán, khách quan và khả năng tái lập (*reproducibility*), tạo nền tảng vững chắc cho quá trình huấn luyện và đánh giá mô hình ở chương tiếp theo.

Chương 3 Xác định tin giả

Chương này bắt đầu bằng việc giới thiệu sơ lược bài toán phát hiện tin giả tiếng Việt cũng như ý tưởng tiếp cận của nhóm cho bài toán này. Sau đó, một số phương pháp trích chọn đặc trưng thủ công tiêu biểu và các mô hình học sâu phổ biến mà nhóm áp dụng, bao gồm các mô hình truyền thống và mô hình ngôn ngữ tiền huấn luyện, sẽ được tóm lược thông qua lịch sử hình thành và nguyên lý hoạt động của chúng.

3.1 Phương pháp đề xuất

Bài toán phát hiện tin giả là một trong những bài toán quan trọng thuộc lĩnh vực xử lý ngôn ngữ tự nhiên, đóng vai trò thiết yếu trong bối cảnh thông tin trực tuyến ngày càng phát triển mạnh mẽ. Sự lan truyền nhanh chóng của các nội dung sai lệch có thể gây ảnh hưởng tiêu cực đến nhận thức cộng đồng và đời sống xã hội. Vì vậy, việc không ngừng nghiên cứu và cải thiện các phương pháp phát hiện tin giả là điều cần thiết.

Cho đến nay, đã có nhiều kỹ thuật trích xuất đặc trưng văn bản được các nhà nghiên cứu đề xuất và áp dụng hiệu quả, chẳng hạn như *Bag-of-Words*, *TF-IDF* hay các biểu diễn dựa trên *word embedding*. Bên cạnh đó, các mô hình học sâu, đặc biệt là các mạng nơ-ron hồi tiếp và các mô hình ngôn ngữ tiền huấn luyện như *Transformer*, cũng chứng minh được tính vượt trội thông qua những kết quả đầy hứa hẹn trên nhiều bộ dữ liệu khác nhau.

Vì thế, để lựa chọn phương pháp phù hợp và hiệu quả nhất cho bài toán phát hiện tin giả tiếng Việt, nhóm quyết định tiến hành so sánh hiệu năng giữa các phương pháp trích chọn đặc trưng thủ công kết hợp với các mô hình phân lớp truyền thống và các mô hình học sâu hiện đại. Đối với các phương pháp trích chọn đặc trưng thủ công, nhóm sẽ triển khai các thuật toán phân lớp như *Logistic Regression*, *SVM* và các mô hình học sâu như *BiLSTM* và *PhoBERT* nhằm đánh giá và phân tích hiệu năng một cách toàn diện.

3.2 Lý do lựa chọn các nhóm mô hình

Trong bài toán phân loại văn bản, đặc biệt là phát hiện tin giả, việc lựa chọn mô hình cần cân bằng giữa ba yếu tố chính:

- Khả năng diễn giải của mô hình
- Chi phí tính toán
- Năng lực biểu diễn ngữ nghĩa

Dựa trên các tiêu chí này, nghiên cứu so sánh hai nhóm phương pháp chính:

1. Nhóm học máy truyền thống

Bao gồm:

- Logistic Regression
- Support Vector Machine

Đầu vào của các mô hình này là đặc trưng TF-IDF [10]. Hai mô hình có nền tảng lý thuyết vững chắc, hoạt động ổn định trên dữ liệu văn bản có đặc trưng chiều cao và thưa, đồng thời thuận lợi cho việc phân tích đóng góp của đặc trưng và kiểm soát hiện tượng quá khớp [2].

Vì vậy, chúng được sử dụng như các *strong baselines* nhằm đánh giá mức cải thiện thực sự của các phương pháp phức tạp hơn.

2. Nhóm học sâu

Bao gồm:

- BiLSTM
- PhoBERT

BiLSTM kế thừa khả năng ghi nhớ phụ thuộc dài hạn của LSTM [11]. Khi kết hợp với cơ chế attention, mô hình có thể tập trung vào các thành phần thông tin quan trọng trong chuỗi văn bản [12].

PhoBERT là mô hình ngôn ngữ tiền huấn luyện dành riêng cho tiếng Việt, được xây dựng trên nền tảng Transformer/RoBERTa [5, 14, 15]. Mô hình này cho phép khai thác ngữ cảnh hai chiều ở mức sâu và thường đạt hiệu năng cao trong các tác vụ phân loại tiếng Việt.

Việc đặt hai nhóm mô hình trong cùng một khung thực nghiệm giúp trả lời ba câu hỏi nghiên cứu chính:

1. Mức độ cạnh tranh của các mô hình truyền thống khi đặc trưng được thiết kế tốt.
2. Lợi ích biên của học biểu diễn ngữ cảnh sâu so với đặc trưng TF-IDF.
3. Giá trị gia tăng của tiền huấn luyện tiếng Việt (PhoBERT) đối với bài toán phát hiện tin giả trong điều kiện dữ liệu thực tế.

3.3 Tổng quan kiến trúc hệ thống

Hệ thống được thiết kế theo ba giai đoạn chính:

1. **Tiền xử lý:** Văn bản được làm sạch nhằm loại bỏ các ký tự không cần thiết, chuẩn hóa dữ liệu và thực hiện phân đoạn từ tiếng Việt. Sau đó, tập dữ liệu được chia thành các tập huấn luyện, kiểm định và kiểm tra theo tỷ lệ phù hợp nhằm đảm bảo tính khách quan trong quá trình đánh giá mô hình.
2. **Trích xuất đặc trưng:** Tùy theo từng mô hình, nhóm áp dụng các phương pháp biểu diễn văn bản khác nhau. Cụ thể, *TF-IDF* được sử dụng cho các mô hình *Logistic Regression* và *SVM*; *Word Embedding* (FastText) được sử dụng cho mô hình *BiLSTM*; và phương pháp mã hóa *BPE* (*Byte-Pair Encoding*) được sử dụng trong mô hình *PhoBERT* nhằm khai thác thông tin ngữ cảnh ở mức độ sâu hơn.
3. **Phân loại:** Bốn mô hình bao gồm *Logistic Regression*, *SVM*, *BiLSTM* và *PhoBERT* được huấn luyện độc lập trên cùng một tập dữ liệu và được đánh giá thông qua các chỉ số như *Accuracy*, *F1-score* và *ROC-AUC*. Dựa trên các kết quả thu được, nhóm tiến hành so sánh và phân tích hiệu năng của từng phương pháp.

3.4 Trích xuất đặc trưng

3.4.1 TF-IDF cho mô hình học máy truyền thống

3.4.1.1 Lý do lựa chọn TF-IDF

Trong các bài toán phân loại văn bản, TF-IDF được xem là một phương pháp biểu diễn đặc trưng cơ bản nhưng hiệu quả, đặc biệt khi kết hợp với các mô hình phân lớp tuyến tính như Hồi quy Logistic (*Logistic Regression*) và Máy vectơ hỗ trợ (*Support Vector Machine*). Phương pháp này biểu diễn văn bản dưới dạng các vector thưa (*sparse vectors*), trong đó mỗi chiều phản ánh mức độ quan trọng của một từ trong văn bản so với toàn bộ tập dữ liệu.

Mặc dù các phương pháp biểu diễn hiện đại như nhúng từ (*word embedding*) hay các mô hình ngôn ngữ tiền huấn luyện có khả năng khai thác ngữ nghĩa sâu hơn, TF-IDF vẫn được sử dụng rộng rãi như một baseline mạnh trong nhiều nghiên cứu phân loại văn bản. Đặc biệt đối với các mô hình tuyến tính, biểu diễn TF-IDF giúp khai thác hiệu quả các tín hiệu từ vựng (*lexical cues*) có tính phân biệt cao trong văn bản.

Trong bài toán phát hiện tin giả, các cụm từ giật tít, các mẫu ngôn ngữ gây chú ý hoặc các từ khóa đặc trưng thường đóng vai trò quan trọng trong việc phân biệt giữa tin thật và tin giả. Các tín hiệu này có thể được nắm bắt hiệu quả thông qua các đặc trưng n-gram của TF-IDF. Vì vậy, TF-IDF kết hợp với Logistic Regression và SVM được sử dụng như một baseline mạnh nhằm so sánh với các phương pháp học sâu trong nghiên cứu này.

3.4.1.2 Cấu hình TF-IDF

Để đảm bảo hiệu quả biểu diễn đặc trưng cho các mô hình học máy truyền thống, cấu hình TF-IDF được lựa chọn dựa trên kết quả *ablation study* (Mục 4.4) nhằm tối ưu hóa hiệu năng phân loại. Cụ thể, các siêu tham số được xác định như sau:

- **Kích thước từ điển:** Số lượng đặc trưng được giới hạn ở 40,000 từ/ngữ phổ biến nhất, giúp giảm nhiễu và kiểm soát độ phức tạp của mô hình.
- **N-gram:** Sử dụng kết hợp đơn từ (unigram) và bigram ($n\text{-gram} = (1, 2)$) nhằm khai thác cả thông tin ngữ nghĩa đơn lẻ và các cụm từ liên tiếp, từ đó tăng khả năng nhận diện các mẫu ngữ cảnh đặc trưng của tin giả.
- **Tần số tài liệu:** Các từ xuất hiện quá ít ($\text{min_df}=5$) hoặc quá phổ biến ($\text{max_df}=0.95$) đều bị loại bỏ, đảm bảo chỉ giữ lại các đặc trưng có giá trị phân biệt cao.
- **TF sublinear:** Áp dụng phép biến đổi $\text{TF} = 1 + \log f_{t,d}$ nhằm giảm ảnh hưởng của các từ xuất hiện nhiều lần trong cùng một văn bản, giúp mô hình tập trung vào sự hiện diện của từ hơn là tần suất tuyệt đối.

Việc lựa chọn các tham số này được thực hiện một cách hệ thống nhằm cân bằng giữa khả năng biểu diễn đặc trưng và nguy cơ quá khớp, đồng thời đảm bảo tính tổng quát hóa của mô hình trên các tập dữ liệu kiểm định và kiểm tra.

3.4.2 Nhúng từ (Word Embedding) cho BiLSTM

Để phục vụ cho mô hình BiLSTM, biểu diễn văn bản được thực hiện thông qua kỹ thuật nhúng từ (word embedding) kết hợp giữa embedding tiền huấn luyện và học từ dữ liệu. Từ điển được xây dựng dựa trên toàn bộ tập train, loại bỏ các từ xuất hiện dưới 2 lần nhằm giảm nhiễu và kiểm soát kích thước không gian từ vựng, kết quả thu được **23,166** từ.

Hai token đặc biệt được bổ sung: <PAD> (index 0) dùng để đệm các chuỗi ngắn và <UNK> (index 1) đại diện cho các từ ngoài từ điển. Mỗi văn bản sau tiền xử lý được

chuyển đổi thành một chuỗi số nguyên với độ dài tối đa 128, đảm bảo tính nhất quán về kích thước đầu vào cho mô hình tuần tự.

Các vector embedding có chiều 300 được khởi tạo từ bộ embedding tiền huấn luyện FastText [8] phiên bản tiếng Việt (`cc.vi.300.bin`). FastText sử dụng thông tin n-gram ký tự để xây dựng biểu diễn từ, cho phép xử lý hiệu quả các từ ngoài từ điển bằng cách tổng hợp vector của các n-gram thành phần. Ma trận embedding tiền huấn luyện được nạp vào lớp embedding của mô hình và tiếp tục được tinh chỉnh trong quá trình huấn luyện thông qua lan truyền ngược, cho phép mô hình điều chỉnh biểu diễn ngữ nghĩa phù hợp với nhiệm vụ phân loại tin giả. Việc kết hợp embedding tiền huấn luyện với fine-tuning giúp tận dụng tri thức ngôn ngữ học được từ ngữ liệu lớn (Common Crawl), đồng thời thích ứng tốt với đặc thù dữ liệu tiếng Việt trong miền bài toán.

3.4.3 Tokenization PhoBERT (BPE)

Quá trình mã hóa đầu vào cho PhoBERT sử dụng thuật toán Byte-Pair Encoding (BPE) [16] với từ điển gồm 64.000 subword, cho phép mô hình xử lý hiệu quả các từ ngoài từ điển và hiện tượng đa hình trong tiếng Việt. Mỗi văn bản được tiền xử lý như sau:

- Thêm token đặc biệt [CLS] ở đầu và [SEP] ở cuối chuỗi để đánh dấu ranh giới văn bản.
- Chuỗi token được cắt hoặc đệm về độ dài cố định 256 nhằm đảm bảo tính nhất quán đầu vào.
- Sinh `attention_mask` để phân biệt token thực (1) và token đệm (0), hỗ trợ quá trình attention trong mô hình.

Chiến lược này giúp PhoBERT tận dụng tối đa thông tin ngữ cảnh và đảm bảo khả năng tổng quát hóa trên dữ liệu tiếng Việt đa dạng.

3.5 Các mô hình phân loại

3.5.1 Các mô hình học máy truyền thống

3.5.1.1 Hồi quy Logistic

Mô hình hồi quy Logistic sử dụng điều chuẩn L2 nhằm hạn chế quá khớp và áp dụng cơ chế cân bằng lớp (`class_weight='balanced'`) để xử lý mất cân bằng nhãn. Các siêu tham số được lựa chọn thông qua GridSearchCV 5-fold, đảm bảo tính khách quan và tổng quát hóa:

- Lưới tìm kiếm: $C \in \{0.5; 1; 5; 10; 20; 50\}$, $\text{solver} \in \{\text{liblinear}, \text{saga}\}$
- Cấu hình tối ưu: $C = 10$, $\text{solver} = \text{saga}$, $\text{max_iter} = 3000$

Việc lựa chọn này giúp mô hình đạt hiệu năng tốt nhất trên tập kiểm định mà không bị phụ thuộc vào phân phối nhãn.

3.5.1.2 Máy vectơ hỗ trợ

Mô hình máy vectơ hỗ trợ (SVM) sử dụng LinearSVC với trọng số lớp cân bằng để giảm thiểu ảnh hưởng của mất cân bằng nhãn. Quá trình tối ưu siêu tham số được thực hiện qua GridSearchCV với 5-fold cross-validation:

- Lưới tìm kiếm: $C \in \{0.1; 0.5; 1; 2; 5; 10\}$
- Cấu hình tối ưu: $C = 0.5$, LinearSVC kết hợp hiệu chuẩn xác suất (calibration)

Chiến lược này đảm bảo SVM vừa tận dụng được đặc trưng TF-IDF vừa duy trì khả năng phân biệt tốt giữa các lớp.

3.5.2 Các mô hình học sâu

Như đã phân tích ở Mục 3.2, BiLSTM và PhoBERT được lựa chọn lần lượt làm đại diện cho nhóm mô hình tuần tự và nhóm mô hình ngôn ngữ tiền huấn luyện. Phần này trình bày chi tiết kiến trúc và cấu hình huấn luyện của hai mô hình.

3.5.2.1 Mạng BiLSTM

Kiến trúc BiLSTM được thiết kế gồm các thành phần chính sau:

1. **Embedding:** Ma trận embedding kích thước $(V \times 300)$, cho phép học biểu diễn ngữ nghĩa từ dữ liệu.
2. **BiLSTM 1 tầng:** Mỗi tầng có $\text{hidden_dim}=128$ cho hai chiều (forward/backward), tổng đầu ra mỗi bước là 2×128 .
3. **Soft Attention:** Cơ chế soft attention tính trọng số α_t cho từng bước thời gian, giúp mô hình tập trung vào các từ/cụm từ quan trọng.
4. **Classifier:** Hai lớp dropout [17] (0.3) xen kẽ với các lớp tuyến tính và ReLU, giúp giảm overfitting và tăng khả năng biểu diễn phi tuyến.

Chiến lược huấn luyện sử dụng AdamW [18] ($\eta = 10^{-3}$, $\text{weight_decay}=10^{-4}$), hàm mất mát CrossEntropy có trọng số lớp để xử lý mất cân bằng, $\text{batch}=64$, tối đa 20 epochs với dừng sớm ($\text{patience}=5$). Ngoài ra, áp dụng ReduceLROnPlateau ($\text{factor}=0.5$, $\text{patience}=2$) để điều chỉnh tốc độ học và gradient clipping ($\text{max_norm}=1.0$) nhằm đảm bảo ổn định quá trình tối ưu.

3.5.2.2 Tinh chỉnh PhoBERT

Kiến trúc PhoBERT fine-tuning gồm hai thành phần chính:

1. **Encoder PhoBERT:** 12 lớp Transformer encoder, đầu ra vector [CLS] 768 chiều đại diện toàn cục cho văn bản.
2. **Classifier Head:** Hai lớp Dropout (0.1) xen kẽ với Linear ($768 \rightarrow 384 \rightarrow 2$) và ReLU, giúp tăng khả năng biểu diễn phi tuyến và giảm overfitting.

Chiến lược huấn luyện sử dụng AdamW [18] ($\eta = 3 \times 10^{-5}$, $\text{weight_decay}=0.01$), Linear Warmup (10%), $\text{batch}=16$, tối đa 8 epochs, gradient accumulation steps=4, gradient clipping ($\text{max_norm}=1.0$) và dừng sớm ($\text{patience}=2$).

3.6 Chiến lược xử lý mất cân bằng nhãn

Mặc dù tỷ lệ nhãn trong bộ dữ liệu (55.6% Thật / 44.4% Giả) không quá chênh lệch, sự mất cân bằng vẫn có thể ảnh hưởng đến khả năng phân loại, đặc biệt đối với lớp thiểu số (tin giả). Để giảm thiểu tác động này, nhóm áp dụng hai chiến lược bổ trợ nhau:

- **Chia phân tầng (*stratified split*):** Dữ liệu được chia thành các tập huấn luyện, kiểm định và kiểm tra theo phương pháp phân tầng, đảm bảo tỷ lệ giữa các lớp được bảo toàn đồng đều trong từng tập. Cách tiếp cận này ngăn ngừa tình trạng mô hình bị thiên lệch do sự phân bố nhãn không đều giữa các tập.
- **Trọng số lớp trong hàm mất mát (*class weighting*):** Mỗi lớp c được gán trọng số nghịch đảo tỷ lệ với tần suất xuất hiện:

$$w_c = \frac{N}{K \cdot N_c}$$

trong đó N là tổng số mẫu, K là số lớp và N_c là số mẫu thuộc lớp c . Trọng số này được tích hợp trực tiếp vào hàm mất mát CrossEntropy, tăng mức phạt cho các mẫu thuộc lớp thiểu số, từ đó cải thiện recall cho lớp tin giả. Đối với các mô hình

truyền thống (LR, SVM), cơ chế tương đương được thực hiện thông qua tham số `class_weight='balanced'` trong scikit-learn [19].

Sự kết hợp hai chiến lược trên giúp đảm bảo quá trình huấn luyện và đánh giá phản ánh đúng thực tế phân phối nhãn, đồng thời giảm thiểu nguy cơ mô hình bỏ sót các trường hợp thuộc lớp thiểu số.

3.7 Kết luận chương

Chương này đã trình bày đầy đủ từ trích xuất đặc trưng đến kiến trúc và tham số huấn luyện của bốn mô hình. Thiết kế cho phép so sánh công bằng trên cùng bộ dữ liệu và cùng quy trình đánh giá. Chương sau trình bày kết quả thực nghiệm.

Chương 4 Kết quả thực nghiệm

4.1 Môi trường thực nghiệm

Thực nghiệm được thực hiện trên hệ thống: GPU NVIDIA (CUDA 12.8), RAM 32 GB, Python 3.14.2, PyTorch 2.10, Transformers 4.44, scikit-learn [19] 1.8.

Bảng 4.1: Siêu tham số các mô hình

Mô hình	Siêu tham số
Logistic Regression	C=10, solver=saga, class_weight=balanced, max_iter=3000
SVM	C=0.5, LinearSVC, class_weight=balanced
BiLSTM	Embedding dim=300, Hidden dim=128 Num layers=1, Dropout=0,3 Learning rate=0,001, Batch size=64 Early stopping patience=5
PhoBERT	Pre-trained: vinai/phobert-base Max length=256, Weight decay=0,01 Learning rate=3e-05, Batch size=16 Epochs=8, Warmup ratio=0,1

Bảng 4.1 tóm tắt siêu tham số cuối cùng sau tối ưu hoá.

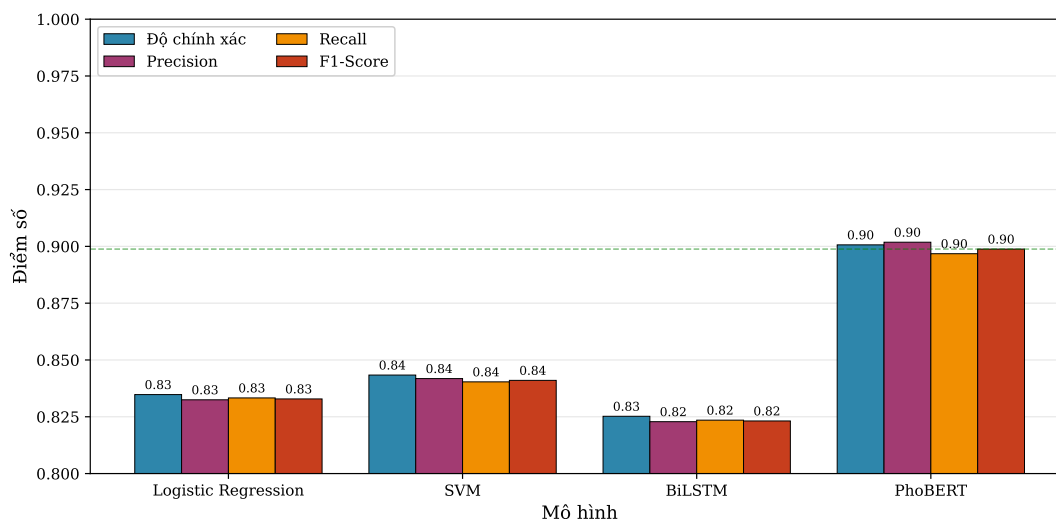
Bảng 4.2: Độ phức tạp và thời gian huấn luyện

Mô hình	Loại	Tham số	Thời gian huấn luyện
Logistic Regression	ML truyền thống	~10K	N/A
SVM	ML truyền thống	~10K	N/A
BiLSTM	Học sâu	~500K	N/A
PhoBERT	Transformer	~135M	N/A

Bảng 4.2 so sánh số tham số và thời gian huấn luyện giữa các mô hình.

4.2 So sánh hiệu năng các mô hình

4.2.1 Kết quả tổng thể



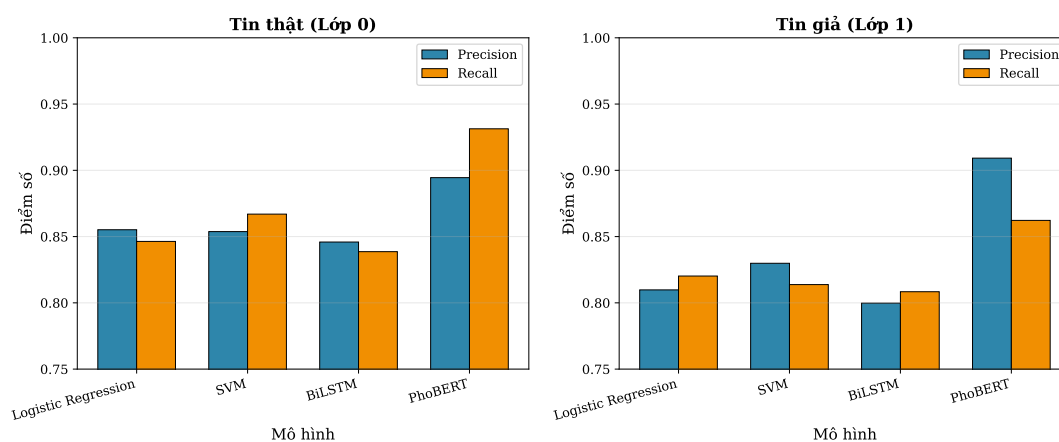
Hình 4.1: So sánh các chỉ số hiệu năng (Accuracy, F1-Score, ROC-AUC) của bốn mô hình trên tập kiểm tra

Bảng 4.3: So sánh hiệu suất các mô hình trên tập kiểm tra

Mô hình	Độ chính xác	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.8348	0.8325	0.8333	0.8329	0.9188
SVM	0.8434	0.8418	0.8404	0.8410	0.9190
BiLSTM	0.8252	0.8228	0.8235	0.8232	0.9048
PhoBERT	0.9007	0.9018	0.8968	0.8988	0.9495

Bảng 4.3 trình bày hiệu năng trên tập kiểm tra. PhoBERT đạt kết quả tốt nhất với **F1 = 89.88%** và **AUC = 0.9495**, vượt trội LR (+6.59%), SVM (+5.78%) và BiLSTM (+7.56%). Đáng chú ý, SVM vượt BiLSTM mặc dù kiến trúc đơn giản hơn, cho thấy đặc trưng TF-IDF chất lượng cao cạnh tranh được với học sâu khi dữ liệu giới hạn.

4.2.2 Hiệu năng theo từng lớp



Hình 4.2: Precision, Recall và F1-Score theo từng lớp (Thật / Giả) cho bốn mô hình

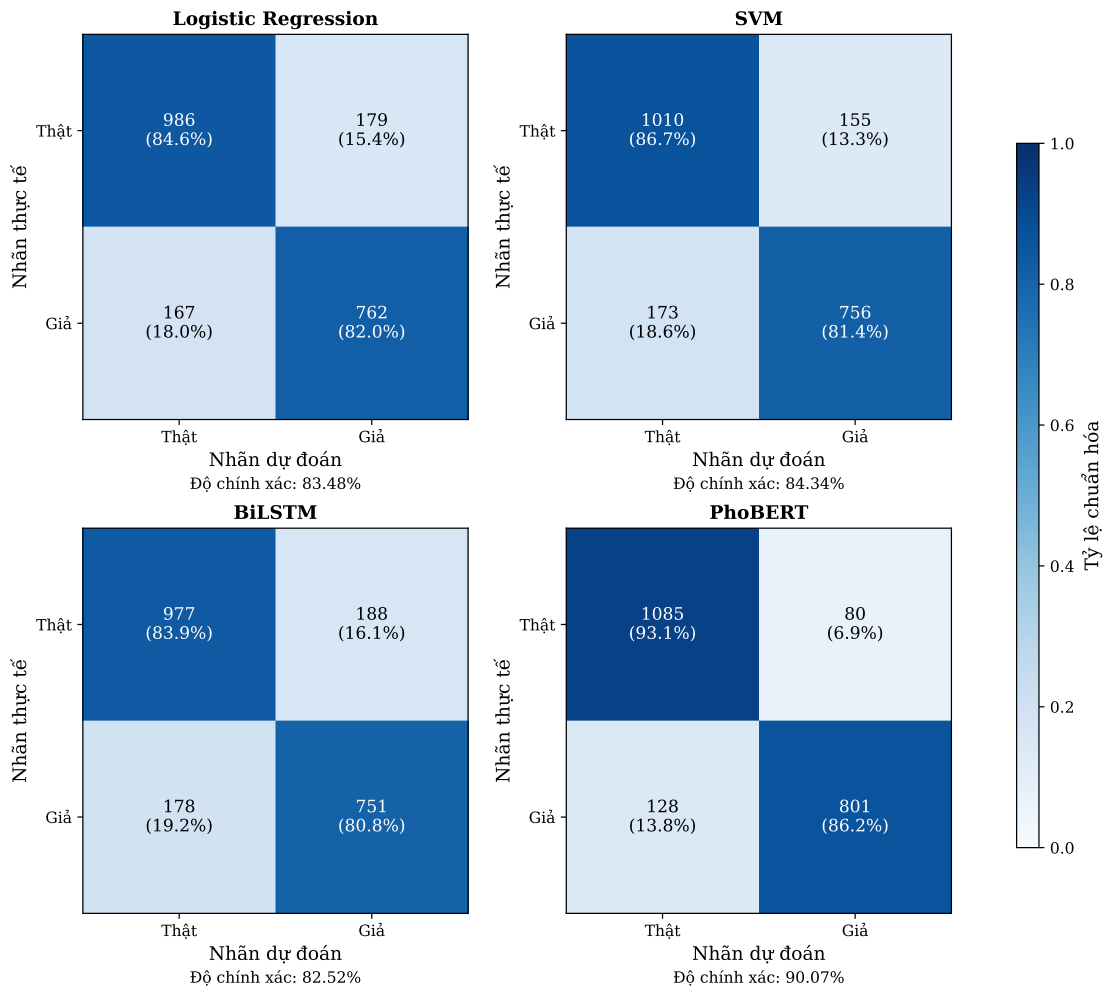
Bảng 4.4: Chỉ số hiệu suất theo từng lớp

Mô hình	Lớp	Precision	Recall	F1-Score
Logistic Regression	Thật	0.8552	0.8464	0.8507
	Giả	0.8098	0.8202	0.8150
SVM	Thật	0.8538	0.8670	0.8603
	Giả	0.8299	0.8138	0.8217
BiLSTM	Thật	0.8459	0.8386	0.8422
	Giả	0.7998	0.8084	0.8041
PhoBERT	Thật	0.8945	0.9313	0.9125
	Giả	0.9092	0.8622	0.8851

Từ Bảng 4.4:

- PhoBERT: Recall = 86.22% cho lớp Giả – phát hiện tin giả tốt nhất trong bốn mô hình.
- BiLSTM: Recall = 80.84% cho lớp Giả – bỏ sót nhiều tin giả nhất (nguy hiểm trong thực tế).
- SVM đạt Recall = 81.38% cho lớp Giả, cao hơn LR (82.02%) và BiLSTM.

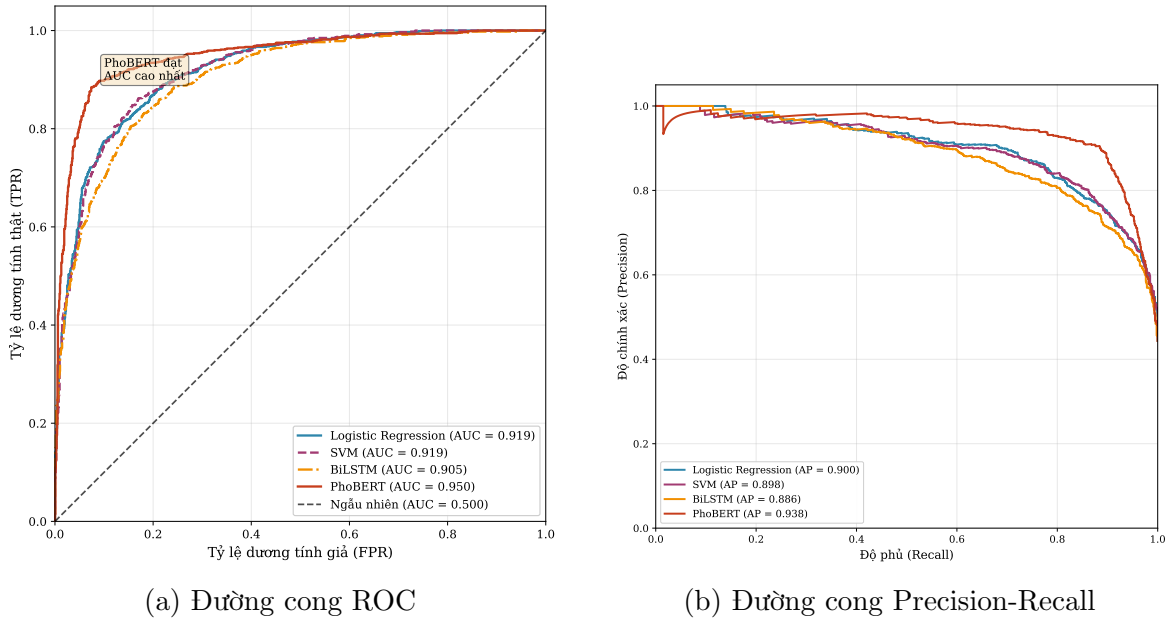
4.2.3 Ma trận nhầm lẫn



Hình 4.3: Ma trận nhầm lẫn của bốn mô hình trên tập kiểm tra (2,094 mẫu)

PhoBERT: 1,085 âm tính thật (True Negative – TN), 801 dương tính thật (True Positive – TP), 80 dương tính giả (False Positive – FP), 128 âm tính giả (False Negative – FN) trên 2,094 mẫu test. BiLSTM có nhiều lỗi nhất (366 lỗi, FP=188, FN=178), trong khi PhoBERT ít lỗi nhất (208 lỗi).

4.2.4 Đường cong ROC và Precision-Recall



Hình 4.4: Đường cong ROC và Precision-Recall của bốn mô hình trên tập kiểm tra

PhoBERT ($AUC = 0.9495$) áp đảo so với SVM (0.9190), LR (0.9188) và BiLSTM (0.9048).

4.2.5 Phân tích hiệu chuẩn xác suất (Calibration Analysis)

Bên cạnh các chỉ số phân loại truyền thống, việc đánh giá chất lượng xác suất dự đoán của mô hình cũng đóng vai trò quan trọng trong các ứng dụng thực tế. Một mô hình được coi là có hiệu chuẩn tốt (*well-calibrated*) khi xác suất dự đoán phản ánh đúng tỷ lệ dương tính thực tế: nếu mô hình dự đoán xác suất 80% cho một nhóm mẫu, thì khoảng 80% số mẫu đó phải thực sự thuộc lớp dương.

Điều này đặc biệt quan trọng trong bài toán phát hiện tin giả, vì người dùng hoặc hệ thống downstream cần biết mức độ tin cậy thực sự của mỗi dự đoán để quyết định hành động phù hợp (ví dụ: đánh dấu cảnh báo, chuyển sang kiểm tra thủ công, hoặc tự động loại bỏ).

Nghiên cứu sử dụng ba chỉ số đánh giá hiệu chuẩn [20]:

- **Expected Calibration Error (ECE):** Sai số hiệu chuẩn kỳ vọng, đo mức chênh lệch trung bình có trọng số giữa xác suất dự đoán và tỷ lệ dương tính thực tế trên

các nhóm mẫu (bin):

$$\text{ECE} = \sum_{b=1}^B \frac{|B_b|}{N} \cdot |\text{acc}(B_b) - \text{conf}(B_b)|$$

trong đó B_b là tập mẫu trong bin thứ b , $\text{acc}(B_b)$ là tỷ lệ dự đoán đúng và $\text{conf}(B_b)$ là xác suất dự đoán trung bình trong bin đó.

- **Maximum Calibration Error (MCE):** Sai số hiệu chuẩn cực đại, phản ánh bin có độ lệch lớn nhất:

$$\text{MCE} = \max_b |\text{acc}(B_b) - \text{conf}(B_b)|$$

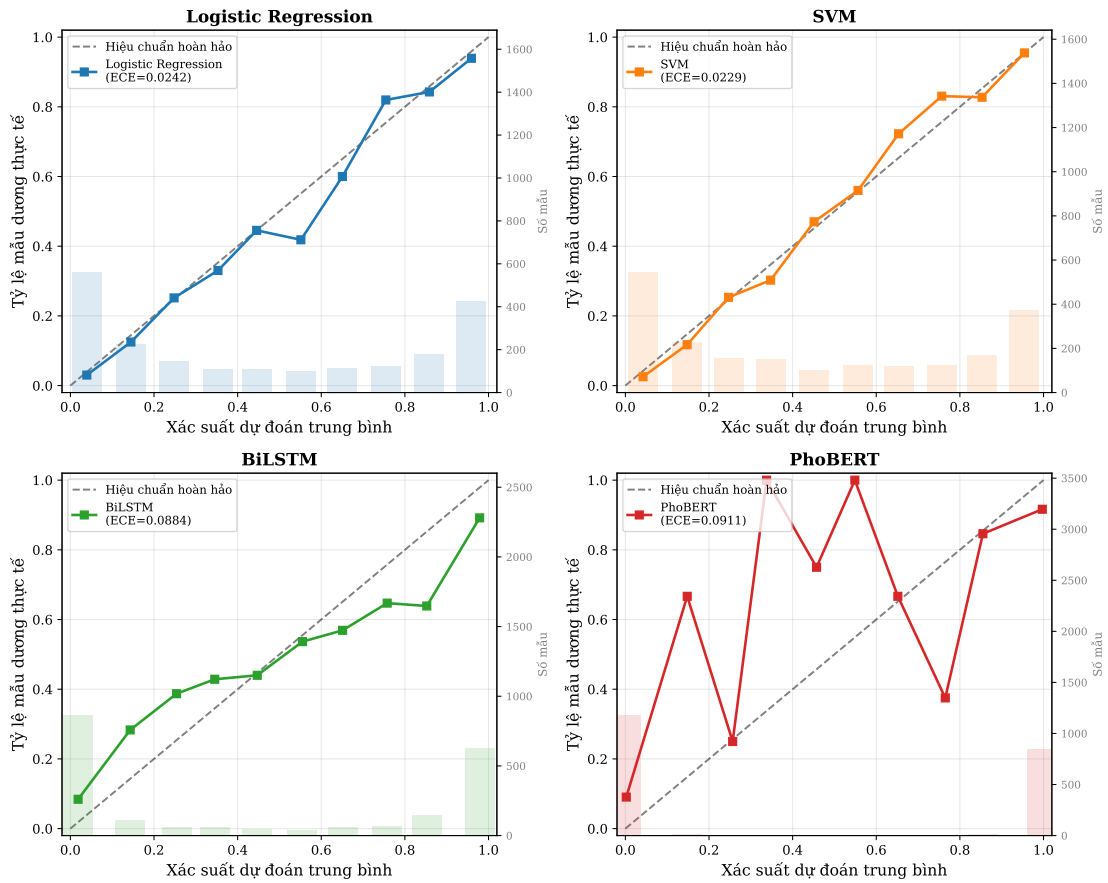
- **Brier Score:** Trung bình bình phương sai số giữa xác suất dự đoán và nhãn thực:

$$\text{BS} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2$$

Brier Score kết hợp cả khả năng phân loại và hiệu chuẩn trong một chỉ số duy nhất.

Cả ba chỉ số đều có giá trị càng thấp càng tốt (0 = hoàn hảo).

Biểu đồ hiệu chuẩn (Reliability Diagram) của bốn mô hình



Hình 4.5: Biểu đồ hiệu chuẩn (Reliability Diagram) của bốn mô hình trên tập kiểm tra. Đường chéo nét đứt biểu diễn hiệu chuẩn hoàn hảo; các điểm vuông biểu diễn tỷ lệ dương tính thực tế trong mỗi bin xác suất.

Bảng 4.5: Chỉ số hiệu chuẩn xác suất của bốn mô hình trên tập kiểm tra

Mô hình	ECE ↓	MCE ↓	Brier Score ↓
Logistic Regression	0.0242	0.1329	0.1136
SVM	0.0229	0.0740	0.1134
BiLSTM	0.0884	0.2135	0.1348
PhoBERT	0.0911	0.6622	0.0925

Từ Bảng 4.5 và Hình 4.5, có thể rút ra một số quan sát đáng chú ý:

- **Các mô hình học máy truyền thống có hiệu chuẩn tốt nhất:** SVM đạt ECE thấp nhất (0.0229) và MCE thấp nhất (0.0740), theo sau là LR (ECE = 0.0242). Điều này cho thấy xác suất dự đoán của SVM và LR phản ánh khá chính xác tỷ

lệ dương tính thực tế. Kết quả này phù hợp với lý thuyết, vì Logistic Regression được thiết kế để ước lượng xác suất trực tiếp, và SVM sử dụng hiệu chuẩn Platt (CalibratedClassifierCV) trong quá trình huấn luyện.

- **PhoBERT có Brier Score tốt nhất nhưng MCE cao:** PhoBERT đạt Brier Score thấp nhất (0.0925), phản ánh khả năng phân loại vượt trội. Tuy nhiên, MCE rất cao (0.6622) cho thấy mô hình có xu hướng *overconfident* – đưa ra xác suất rất cao (gần 0 hoặc 1) ngay cả khi dự đoán sai. Hiện tượng này là đặc trưng phổ biến của các mô hình Transformer, đặc biệt sau quá trình fine-tuning trên tập dữ liệu nhỏ.
- **BiLSTM có hiệu chuẩn kém nhất:** Với $ECE = 0.0884$ và Brier Score = 0.1348, BiLSTM cho thấy xác suất dự đoán không phản ánh tốt độ chính xác thực tế, nhất quán với hiệu năng phân loại thấp nhất trong bốn mô hình.

Hàm ý thực tế. Phân tích hiệu chuẩn cho thấy trong các hệ thống triển khai thực tế, *không nên sử dụng trực tiếp* xác suất thô (raw probability) của PhoBERT làm mức độ tin cậy. Thay vào đó, cần áp dụng kỹ thuật hiệu chuẩn hậu xử lý (*post-hoc calibration*) như Temperature Scaling hoặc Platt Scaling để cải thiện chất lượng xác suất trước khi đưa vào hệ thống ra quyết định. Ngược lại, xác suất từ SVM và LR có thể được sử dụng trực tiếp với độ tin cậy cao hơn.

4.3 Kiểm định thống kê

4.3.1 Kiểm định McNemar

Kiểm định McNemar [21] được sử dụng để so sánh hiệu năng của hai mô hình phân loại trên cùng một tập kiểm thử, dựa trên các dự đoán theo từng mẫu (paired predictions). Kiểm định này đặc biệt phù hợp trong bối cảnh hai mô hình được đánh giá trên cùng dữ liệu và cần xác định liệu sự khác biệt quan sát được có mang ý nghĩa thống kê hay chỉ do ngẫu nhiên.

Giả thuyết kiểm định được thiết lập như sau:

- H_0 : Không tồn tại sự khác biệt có ý nghĩa thống kê giữa xác suất dự đoán đúng của hai mô hình.
- H_1 : Tồn tại sự khác biệt có ý nghĩa thống kê giữa xác suất dự đoán đúng của hai mô hình.

Thống kê kiểm định được tính theo công thức:

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}$$

Hai đại lượng b và c đại diện cho các trường hợp bất đồng (discordant pairs) giữa hai mô hình. Các trường hợp cả hai cùng đúng hoặc cùng sai không ảnh hưởng đến thống kê kiểm định vì không phản ánh sự khác biệt tương đối.

Giá trị χ^2 càng lớn cho thấy mức độ chênh lệch giữa b và c càng cao, tức là sự khác biệt có xu hướng mang tính hệ thống thay vì ngẫu nhiên. Giá trị p -value được tính dựa trên phân phối Chi-square với 1 bậc tự do. Nếu $p < \alpha$ (với $\alpha = 0.05$), giả thuyết H_0 bị bác bỏ và có thể kết luận sự khác biệt giữa hai mô hình là có ý nghĩa thống kê.

Hệ số điều chỉnh (-1) trong tử số là hiệu chỉnh liên tục (*continuity correction*), giúp giảm sai số loại I khi kích thước mẫu không quá lớn.

Do nghiên cứu thực hiện sáu phép so sánh cặp mô hình, việc tiến hành nhiều kiểm định đồng thời có thể làm gia tăng xác suất sai loại I. Để kiểm soát mức sai số toàn cục (*family-wise error rate*), phương pháp hiệu chỉnh Holm–Bonferroni được áp dụng.

Cụ thể, các p -value được sắp xếp theo thứ tự tăng dần và so sánh tuần tự với các ngưỡng hiệu chỉnh tương ứng, bảo đảm rằng xác suất xuất hiện ít nhất một kết luận sai không vượt quá mức ý nghĩa α đã lựa chọn.

Với hiệu chỉnh Holm–Bonferroni [22] cho 6 phép so sánh:

Bảng 4.6: Kết quả kiểm định McNemar (sau hiệu chỉnh Holm–Bonferroni)

Mô hình 1	Mô hình 2	χ^2	p -value	Kết luận
LR	SVM	4.52	0.0945	Không có ý nghĩa
LR	BiLSTM	1.24	0.2645	Không có ý nghĩa
LR	PhoBERT	57.22	<0.001	Có ý nghĩa (***)
SVM	BiLSTM	4.63	0.0945	Không có ý nghĩa
SVM	PhoBERT	45.68	<0.001	Có ý nghĩa (***)
BiLSTM	PhoBERT	66.62	<0.001	Có ý nghĩa (***)

PhoBERT vượt trội có ý nghĩa thống kê so với cả ba mô hình còn lại ($p < 0.001$). LR, SVM và BiLSTM không khác biệt đáng kể với nhau ($p \geq 0.05$ sau hiệu chỉnh Holm–Bonferroni) dù kiến trúc khác nhau hoàn toàn.

4.3.2 Khoảng tin cậy Bootstrap

Bên cạnh kiểm định McNemar, nghiên cứu tiếp tục sử dụng phương pháp Bootstrap nhằm ước lượng độ ổn định của chỉ số F1-score và đánh giá độ tin cậy của sự khác biệt

giữa các mô hình.

Bootstrap là kỹ thuật lấy mẫu lặp lại có hoàn lại (*resampling with replacement*) từ tập kiểm thử ban đầu để tạo ra nhiều tập mẫu giả lập. Trong nghiên cứu này, quá trình lấy mẫu được thực hiện 10,000 lần. Với mỗi lần lấy mẫu, F1-score được tính lại, từ đó xây dựng phân phối thực nghiệm của chỉ số này.

Khoảng tin cậy 95% (95% Confidence Interval – CI) được xác định từ phân vị 2.5% và 97.5% của phân phối Bootstrap. Khoảng này biểu diễn miền giá trị mà tham số thực (F1-score kỳ vọng trên tổng thể) có khả năng nằm trong đó với mức tin cậy 95%.

Việc sử dụng Bootstrap mang ý nghĩa quan trọng trong đánh giá thực nghiệm vì:

- Không giả định phân phối chuẩn của F1-score.
- Phản ánh mức độ dao động của mô hình khi dữ liệu thay đổi.
- Cho phép so sánh mức độ chồng lấp giữa các khoảng tin cậy, từ đó hỗ trợ xác nhận sự khác biệt có tính hệ thống.

Kết quả khoảng tin cậy 95% cho các mô hình được trình bày trong bảng 4.7.

Bảng 4.7: Khoảng tin cậy 95% Bootstrap [1] cho F1-Score (10,000 lần lấy mẫu lại)

Mô hình	F1-Score	95% CI
LR	83.29	[0.8165; 0.8487]
SVM	84.10	[0.8253; 0.8564]
BiLSTM	82.32	[0.8063; 0.8391]
PhoBERT	89.88	[0.8853; 0.9114]

Từ Bảng 4.7 có thể quan sát rằng:

- PhoBERT đạt F1-score trung bình cao nhất (89.88%) với khoảng tin cậy 95% là [0.8853; 0.9114].
- Khoảng tin cậy của PhoBERT không chồng lấp với bất kỳ mô hình nào còn lại.
- Các mô hình LR, SVM và BiLSTM có khoảng tin cậy tương đối gần nhau và tồn tại hiện tượng chồng lấp.

Sự không chồng lấp khoảng tin cậy giữa PhoBERT và các mô hình khác củng cố kết luận từ kiểm định McNemar rằng sự vượt trội của PhoBERT không phải do dao động ngẫu nhiên mà mang tính ổn định thống kê. Ngược lại, sự chồng lấp giữa LR, SVM và

BiLSTM cho thấy sự khác biệt hiệu năng giữa ba mô hình này chưa đủ mạnh để kết luận tồn tại ưu thế rõ ràng.

Như vậy, Bootstrap đóng vai trò bổ sung cho kiểm định McNemar: nếu McNemar kiểm tra sự khác biệt ở mức dự đoán từng mẫu (*instance-level comparison*), thì Bootstrap đánh giá độ ổn định của chỉ số tổng hợp (F1-score) dưới biến thiên dữ liệu (*metric-level stability*). Việc kết hợp cả hai phương pháp giúp tăng độ tin cậy và tính thuyết phục cho kết luận thực nghiệm.

4.3.3 Kiểm tra chéo (Cross-Validation)

Bên cạnh việc đánh giá trên tập kiểm thử cố định, nghiên cứu tiếp tục thực hiện kiểm tra chéo phân tầng (*Stratified Cross-Validation*) nhằm đánh giá tính ổn định của mô hình dưới các cách chia dữ liệu khác nhau.

Cụ thể, phương pháp 5-fold cross-validation được áp dụng với 3 seed ngẫu nhiên khác nhau, tạo thành tổng cộng 15 lần huấn luyện và đánh giá. Việc lặp lại trên nhiều seed giúp giảm ảnh hưởng của tính ngẫu nhiên trong quá trình chia dữ liệu và cung cấp ước lượng ổn định hơn về hiệu năng trung bình.

Trong mỗi lần lặp, mô hình được huấn luyện trên 4 folds và kiểm tra trên fold còn lại. Các chỉ số Accuracy và F1-score được ghi nhận cho từng lần chạy, sau đó tính giá trị trung bình và độ lệch chuẩn.

Độ lệch chuẩn ($\pm$) phản ánh mức độ dao động của hiệu năng giữa các lần chia dữ liệu. Giá trị này càng nhỏ cho thấy mô hình càng ổn định và ít phụ thuộc vào cách phân chia tập dữ liệu. Do hạn chế về tài nguyên tính toán và thời gian huấn luyện đáng kể của các mô hình học sâu như BiLSTM và PhoBERT, quy trình kiểm tra chéo 5-fold với 3 seed (tổng 15 lần chạy) chỉ được áp dụng cho các mô hình học máy truyền thống (Logistic Regression và SVM). Việc mở rộng quy trình này cho các mô hình học sâu sẽ đòi hỏi chi phí tính toán rất lớn và vượt quá phạm vi thực nghiệm của nghiên cứu.

Kết quả kiểm tra chéo được trình bày trong Bảng 4.8.

Bảng 4.8: Kết quả kiểm tra chéo 5-fold (3 seeds)

Mô hình	Accuracy	F1-Score
Logistic Regression	0.8301 ± 0.0087	0.8282 ± 0.0087
SVM	0.8490 ± 0.0087	0.8469 ± 0.0087

Từ Bảng 4.8 có thể quan sát rằng:

- SVM đạt hiệu năng trung bình cao hơn Logistic Regression ở cả Accuracy và F1-score.

- Độ lệch chuẩn của cả hai mô hình đều nhỏ (≤ 0.010), cho thấy hiệu năng ổn định qua các lần chia dữ liệu.
- Mức dao động thấp xác nhận rằng kết quả không phụ thuộc đáng kể vào một cấu hình phân chia cụ thể.

So với kiểm định McNemar và Bootstrap, Cross-Validation bổ sung một góc nhìn khác trong đánh giá thực nghiệm: nếu McNemar kiểm tra sự khác biệt ở mức dự đoán từng mẫu (*instance-level comparison*) và Bootstrap đánh giá độ ổn định của chỉ số trên tập kiểm thử cố định, thì Cross-Validation kiểm tra khả năng tổng quát hóa của mô hình khi dữ liệu huấn luyện thay đổi.

Việc kết hợp ba phương pháp đánh giá (McNemar, Bootstrap và Cross-Validation) giúp đảm bảo rằng kết luận về hiệu năng mô hình không chỉ dựa trên một lần thử nghiệm duy nhất, mà được xác nhận dưới nhiều góc độ thống kê khác nhau.

4.4 Nghiên cứu triệt bỏ (Ablation Study)

Ablation Study là phương pháp thực nghiệm nhằm đánh giá mức độ đóng góp của từng thành phần trong một hệ thống bằng cách thay đổi hoặc loại bỏ từng yếu tố riêng lẻ trong khi giữ nguyên các thành phần còn lại. Mục tiêu của phương pháp này không chỉ là tìm cấu hình tốt nhất, mà còn xác định thành phần nào thực sự tạo ra cải thiện hiệu năng.

Trong nghiên cứu này, Ablation Study được thực hiện trên pipeline TF-IDF + Logistic Regression bằng cách lần lượt thay đổi các yếu tố sau:

- Kích thước từ điển
- Cấu hình n -gram
- Phương pháp phân đoạn từ
- Tham số điều chuẩn C
- Phương pháp chuẩn hóa TF

Mỗi yếu tố được thay đổi độc lập trong khi giữ nguyên các thành phần còn lại, nhằm đảm bảo có thể đánh giá riêng tác động của từng thành phần lên các chỉ số Accuracy và F1-score. Cách tiếp cận này cho phép xác định mức độ nhạy cảm của mô hình đối với từng cấu hình và làm rõ yếu tố nào đóng vai trò quyết định trong việc cải thiện hiệu năng.

Do cấu trúc pipeline TF-IDF + Logistic Regression cho phép tách biệt rõ ràng từng thành phần (kích thước từ điển, cấu hình n -gram, tham số điều chuẩn C , phương pháp chuẩn hóa TF), phương pháp ablation study được áp dụng chủ yếu cho mô hình này nhằm phân tích đóng góp riêng lẻ của từng yếu tố.

Đối với các mô hình học sâu như BiLSTM và PhoBERT, biểu diễn đặc trưng và quá trình học được tích hợp chặt chẽ trong kiến trúc mạng, khiến việc loại bỏ hoặc thay đổi từng thành phần riêng lẻ trở nên phức tạp và khó diễn giải. Việc điều chỉnh một thành phần (ví dụ: embedding, số tầng, chiều ẩn hoặc cơ chế attention) thường kéo theo thay đổi trong toàn bộ động học huấn luyện, do đó không đảm bảo tính so sánh độc lập như trong pipeline tuyến tính.

Bên cạnh đó, chi phí tính toán và thời gian huấn luyện lớn cũng hạn chế khả năng thực hiện ablation toàn diện cho các mô hình này trong phạm vi nghiên cứu. Vì vậy, ablation study được tập trung vào mô hình truyền thống nhằm đảm bảo tính minh bạch, khả năng diễn giải và hiệu quả thực nghiệm.

Kết quả thực nghiệm được trình bày trong Bảng 4.9.

Bảng 4.9: Kết quả Ablation Study trên tập kiểm tra

Thành phần	Cấu hình	Accuracy	F1-Score
Kích thước từ điển	1,000 từ	0.7841	0.7819
	5,000 từ	0.8233	0.8214
	10,000 từ	0.8324	0.8303
	20,000 từ	0.8386	0.8368
	50,000 từ	0.8395	0.8378
N-gram	Unigram only	0.8276	0.8256
	Uni + Bigram	0.8429	0.8410
	Uni+Bi+Trigram	0.8415	0.8397
	Bigram only	0.8152	0.8120
Phân đoạn từ	Có phân đoạn	0.8429	0.8410
	Không phân đoạn	0.8319	0.8300
Giá trị C (LR)	$C = 0.01$	0.7713	0.7679
	$C = 0.1$	0.7942	0.7914
	$C = 1$	0.8429	0.8410
	$C = 10$	0.8367	0.8348
	$C = 100$	0.8357	0.8336
TF Scaling	Standard TF	0.8386	0.8367
	Sublinear TF	0.8429	0.8410

Phân tích và ý nghĩa

1. Kích thước từ điển

Khi tăng kích thước từ điển từ 1,000 lên 10,000 từ, F1-score cải thiện khoảng +4.84%, cho thấy việc mở rộng không gian đặc trưng giúp mô hình khai thác tốt hơn thông tin ngữ nghĩa. Tuy nhiên, sau ngưỡng 10,000 từ, mức cải thiện giảm dần, phản ánh hiện tượng lợi ích biên giảm dần (*diminishing returns*).

Điều này giúp xác định ngưỡng tối ưu giữa độ biểu diễn và độ phức tạp.

2. Cấu hình n -gram

Cấu hình Uni + Bigram đạt hiệu năng tốt nhất. Việc bổ sung bigram giúp mô hình nắm bắt quan hệ ngữ cảnh cục bộ mà unigram đơn lẻ không thể biểu diễn đầy đủ. Tuy nhiên, thêm trigram không mang lại cải thiện đáng kể, có thể do hiện tượng thưa dữ liệu (*sparsity*) tăng cao.

Ablation cho thấy bigram là thành phần đóng góp quan trọng.

3. Phân đoạn từ

Phân đoạn từ không tạo khác biệt lớn đối với TF-IDF + Logistic Regression ($\Delta \approx 1.10\%$), cho thấy mô hình tuyến tính ít phụ thuộc vào cấu trúc ngữ cảnh sâu. Điều này khác với các mô hình ngữ cảnh như PhoBERT, nơi phân đoạn từ có vai trò quan trọng hơn.

Kết quả giúp làm rõ đặc tính từng nhóm mô hình.

4. Tham số điều chuẩn C

Giá trị $C = 1$ cho hiệu năng tốt nhất.

- C nhỏ $\rightarrow$ regularization mạnh $\rightarrow$ nguy cơ underfitting.
- C lớn $\rightarrow$ regularization yếu $\rightarrow$ nguy cơ overfitting.

Ablation xác định điểm cân bằng giữa bias và variance.

5. Chuẩn hóa TF

Sublinear TF nhỉnh hơn Standard TF (+0.43% F1), cho thấy log-scaling mang lại lợi ích nhẹ trên tập dữ liệu hiện tại.

Vậy Ablation Study đã giúp cải tiến gì?

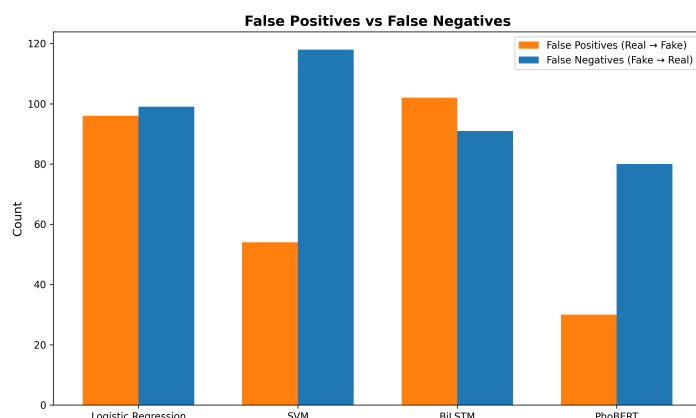
1. Xác định cấu hình tối ưu cho pipeline TF-IDF + Logistic Regression.
2. Loại bỏ các thành phần không đóng góp đáng kể.
3. Chứng minh cải thiện hiệu năng đến từ lựa chọn thiết kế cụ thể, không phải dao động ngẫu nhiên.
4. Cung cấp cơ sở thực nghiệm cho cấu hình sử dụng trong so sánh mô hình ở Mục 4.2.

Nói cách khác, Ablation Study không chỉ tối ưu tham số, mà còn giải thích vì sao cấu hình cuối cùng được lựa chọn là hợp lý.

4.5 Phân tích lỗi

4.5.1 Thống kê lỗi tổng quan

4.5.1.1 So sánh FP và FN



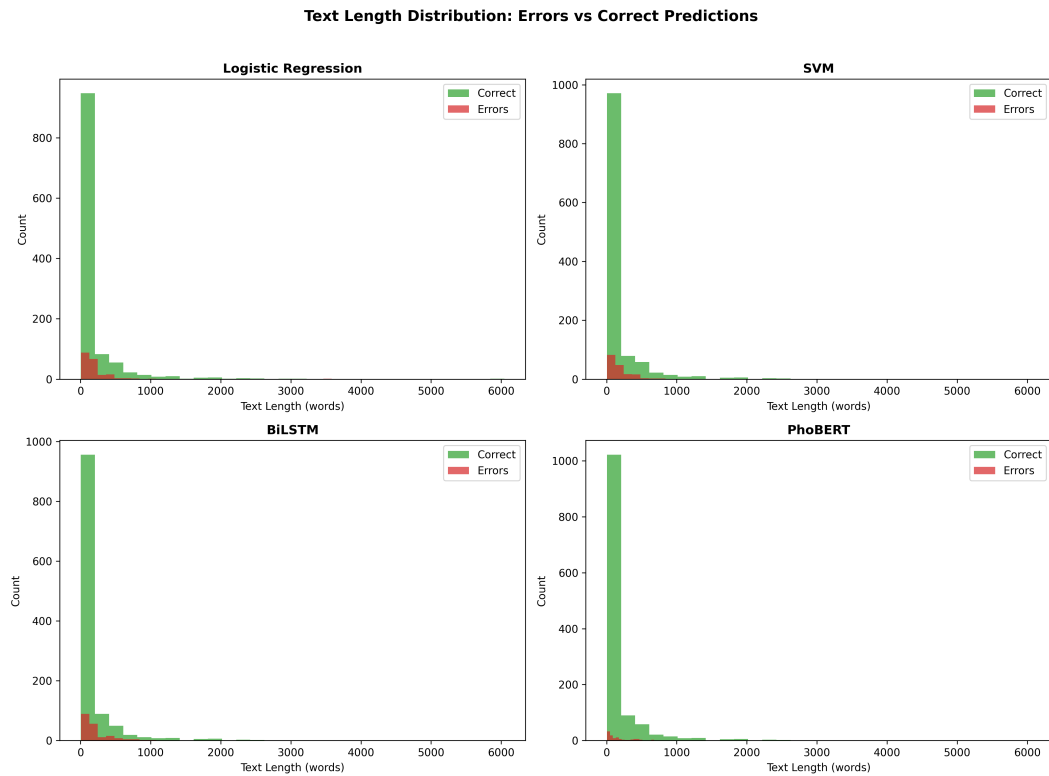
Hình 4.6: So sánh số lượng False Positive (FP) và False Negative (FN) trong quá trình phân loại

Trong Hình 4.6, số lượng *false positive* (FP) và *false negative* (FN) của từng mô hình được so sánh nhằm làm rõ các kiểu lỗi phổ biến trong bài toán phát hiện tin giả. Có thể quan sát rằng các mô hình khác nhau thể hiện xu hướng lỗi khác nhau. Logistic Regression tạo ra số lượng FP và FN khá cân bằng, trong khi SVM có xu hướng tạo ra nhiều FN hơn FP. Điều này cho thấy SVM thường bỏ sót một số trường hợp tin giả, đặc biệt khi các mẫu này không chứa các từ khóa đặc trưng rõ ràng.

Đối với BiLSTM, số lượng lỗi tổng thể cao hơn so với các mô hình còn lại, với cả FP và FN đều ở mức tương đối lớn. Điều này cho thấy mô hình tuần tự này chưa khai thác được đầy đủ thông tin ngữ nghĩa trong văn bản, có thể do kích thước dữ liệu huấn luyện chưa đủ lớn để học các biểu diễn ngữ cảnh phức tạp. Ngược lại, PhoBERT đạt số lượng lỗi thấp nhất trong bốn mô hình, phản ánh khả năng khai thác ngữ cảnh tốt hơn nhờ được tiền huấn luyện trên lượng lớn dữ liệu tiếng Việt.

Phân tích FP và FN cũng cho thấy một số đặc điểm của bài toán. Các FP thường xuất hiện ở những bài viết có tiêu đề mang tính giật gân hoặc chứa các từ khóa thường xuất hiện trong tin giả, khiến mô hình nhầm lẫn và dự đoán sai. Trong khi đó, các FN chủ yếu xảy ra ở những tin giả được viết theo phong cách trung lập hoặc có cấu trúc tương tự các bài báo chính thống, khiến mô hình khó phân biệt.

4.5.1.2 Phân phối lỗi theo độ dài văn bản



Hình 4.7: Phân phối lỗi phân loại theo độ dài văn bản

Hình 4.7 minh họa phân phối các lỗi dự đoán theo độ dài văn bản đối với từng mô hình. Có thể thấy rằng phần lớn các lỗi xảy ra ở các văn bản có độ dài tương đối ngắn. Những văn bản ngắn thường thiếu ngữ cảnh và thông tin bổ sung cần thiết để mô hình có thể xác định chính xác tính xác thực của nội dung. Do đó, cả các mô hình học máy truyền thống và mô hình học sâu đều gặp khó khăn khi xử lý các mẫu này.

Ngược lại, khi độ dài văn bản tăng lên, tỷ lệ lỗi có xu hướng giảm. Các văn bản dài thường chứa nhiều thông tin ngữ nghĩa hơn, giúp mô hình nhận diện các mẫu đặc trưng liên quan đến tin thật hoặc tin giả một cách rõ ràng hơn. Xu hướng này đặc biệt rõ rệt đối với PhoBERT, khi mô hình tận dụng tốt hơn thông tin ngữ cảnh dài nhờ kiến trúc Transformer.

Phân tích này cho thấy độ dài văn bản là một yếu tố ảnh hưởng đáng kể đến hiệu năng của mô hình. Trong các ứng dụng thực tế, các bài viết ngắn hoặc tiêu đề đơn lẻ có thể cần được xử lý bằng các kỹ thuật bổ sung, chẳng hạn như kết hợp thông tin ngữ cảnh từ nguồn dữ liệu khác hoặc sử dụng các đặc trưng bổ sung ngoài nội dung văn bản.

4.5.1.3 Hàm ý cải tiến mô hình

Bảng 4.10: Phân tích lỗi trên tập kiểm tra

Mô hình	Tổng lỗi	Tỉ lệ lỗi	FP	FN
Logistic Regression	346	16.52%	179	167
SVM	328	15.66%	155	173
BiLSTM	366	17.48%	188	178
PhoBERT	208	9.93%	80	128

Bảng 4.10 tổng hợp số lượng lỗi và phân bố FP/FN của từng mô hình trên tập kiểm tra.

Từ các kết quả phân tích lỗi trên, có thể rút ra một số hướng cải tiến tiềm năng cho hệ thống phát hiện tin giả. Thứ nhất, việc kết hợp các đặc trưng ngữ nghĩa sâu với các đặc trưng bề mặt như từ khóa hoặc cụm từ đặc trưng có thể giúp giảm các trường hợp *false positive* trong các mô hình học máy truyền thống. Thứ hai, các mô hình học sâu có thể được cải thiện bằng cách bổ sung thêm dữ liệu huấn luyện hoặc áp dụng các chiến lược tăng cường dữ liệu nhằm giúp mô hình học tốt hơn các mẫu tin giả được viết theo phong cách trung lập.

Ngoài ra, phân tích cũng cho thấy các văn bản ngắn là một nguồn gây lỗi phổ biến. Điều này gợi ý rằng các phương pháp khai thác ngữ cảnh mở rộng, chẳng hạn như kết hợp tiêu đề với nội dung bài viết hoặc tích hợp thông tin từ nguồn bên ngoài, có thể giúp cải thiện khả năng dự đoán đối với các trường hợp khó.

4.5.1.4 Phân tích các trường hợp khó (Hard Examples)

Mặc dù các phân tích trên đã chỉ ra một số nguyên nhân phổ biến gây ra lỗi phân loại, kết quả thực nghiệm cho thấy vẫn tồn tại một nhóm trường hợp đặc biệt khó mà tất cả các mô hình đều thất bại. Những trường hợp này phản ánh các giới hạn hiện tại của các phương pháp chỉ dựa trên nội dung văn bản và cung cấp thêm góc nhìn sâu hơn về những thách thức của bài toán phát hiện tin giả.

Trong quá trình phân tích lỗi, nhóm phát hiện **73 mẫu khó** (*hard examples*) mà tất cả bốn mô hình đều dự đoán sai. Việc kiểm tra thủ công nội dung của các mẫu này cho thấy chúng thường thuộc một số dạng đặc trưng, phản ánh những thách thức phổ biến trong bài toán phát hiện tin giả. Cụ thể, các trường hợp này có thể được chia thành ba nhóm chính: *semantic reasoning*, *knowledge verification*, và *dataset ambiguity*.

Trước hết, một số mẫu yêu cầu suy luận ngữ nghĩa (*semantic reasoning*) để xác định mối quan hệ giữa hai câu. Trong các trường hợp này, hai văn bản có thể không

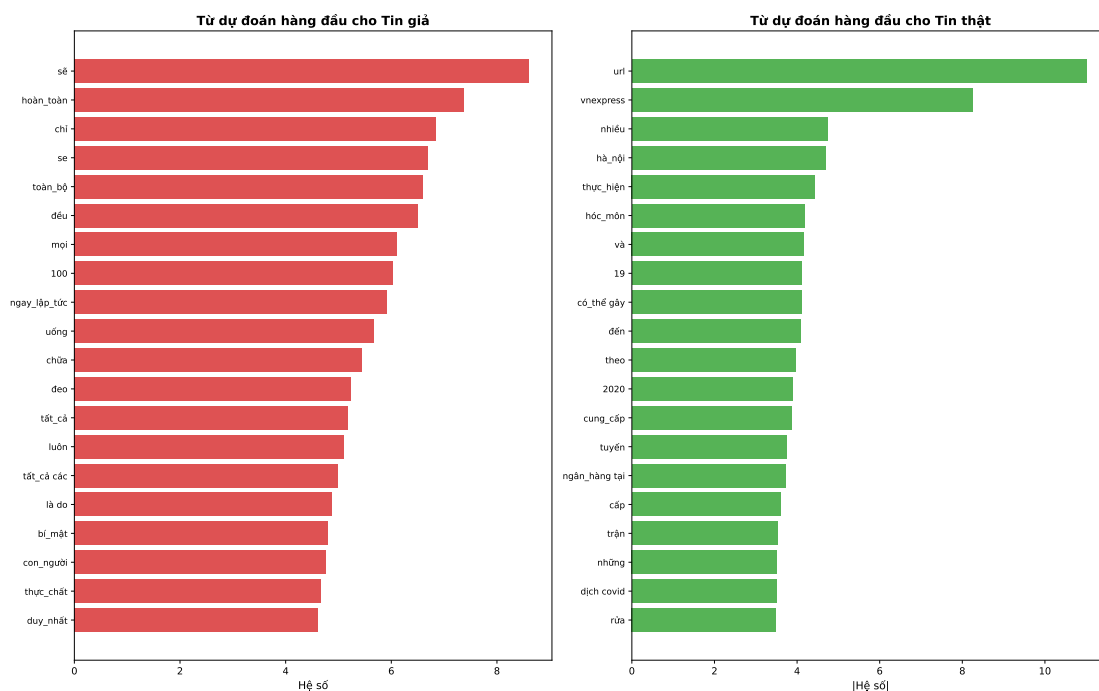
liên quan về mặt nội dung nhưng lại có phong cách viết tương tự, khiến mô hình khó phân biệt. Ví dụ, trong một mẫu dữ liệu, câu “Uống đủ nước mỗi ngày hỗ trợ quá trình trao đổi chất của cơ thể.” được ghép với câu “Ty phú Phạm Nhật Vượng đã quyết định tặng 1000 xe VinFast VF9 cho các y bác sĩ chống dịch Covid-19.” Hai câu này không có quan hệ ngữ nghĩa trực tiếp nhưng đều mang phong cách thông tin trung lập giống các bản tin, dẫn đến việc mô hình nhầm lẫn khi phân loại.

Thứ hai, một số trường hợp thuộc nhóm *knowledge verification*, trong đó việc xác định tính đúng sai của thông tin đòi hỏi kiến thức thực tế ngoài văn bản. Ví dụ, câu “Máy tính cá nhân đầu tiên được bán ra thị trường vào thế kỷ 18.” là một khẳng định sai về mặt lịch sử công nghệ. Tuy nhiên, nếu chỉ dựa vào đặc trưng ngôn ngữ trong văn bản, mô hình khó có thể nhận biết được tính sai lệch của thông tin này. Điều này cho thấy các mô hình hiện tại chủ yếu học các mẫu ngôn ngữ bề mặt và chưa thể thực hiện kiểm chứng tri thức.

Cuối cùng, một số lỗi xuất phát từ *dataset ambiguity*, tức là những trường hợp mà nội dung văn bản không cung cấp đủ ngữ cảnh để xác định nhãn một cách rõ ràng. Chẳng hạn, câu “Hà Nội: học sinh từ cấp 2 trở lên dự kiến đi học trở lại từ 4/5.” có hình thức giống một bản tin chính thống và không chứa dấu hiệu rõ ràng của thông tin sai lệch. Trong các trường hợp như vậy, việc phân loại có thể phụ thuộc vào bối cảnh thời gian hoặc nguồn tin, điều mà các mô hình chỉ dựa trên văn bản không thể khai thác.

Những quan sát này cho thấy các lỗi còn lại không chỉ xuất phát từ hạn chế của mô hình mà còn phản ánh độ phức tạp của bài toán phát hiện tin giả. Các hệ thống trong tương lai có thể được cải thiện bằng cách tích hợp thêm nguồn tri thức bên ngoài hoặc các cơ chế suy luận ngữ nghĩa sâu hơn.

4.5.2 Phân tích khả năng giải thích

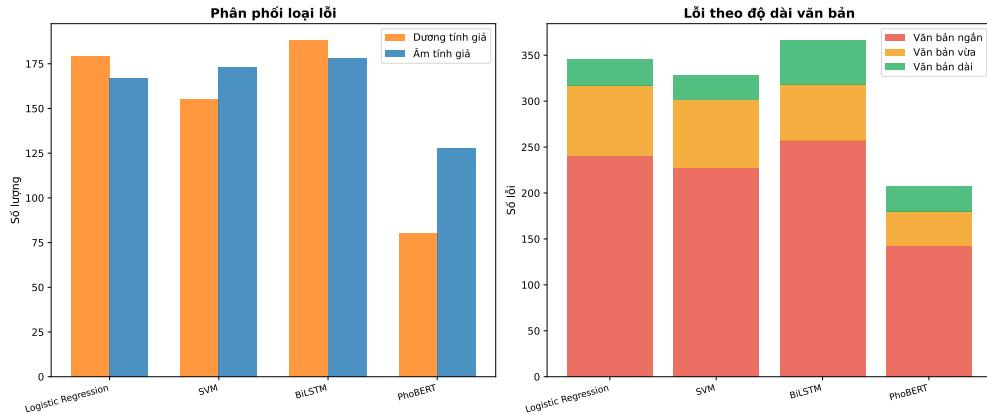


Hình 4.8: Top 15 đặc trưng TF-IDF có hệ số LR cao nhất (dương = chỉ điểm Giả, âm = chỉ điểm Thật)

Hình 4.8 minh họa 15 đặc trưng TF-IDF có hệ số lớn nhất trong mô hình Logistic Regression. Các đặc trưng được chia thành hai nhóm: các đặc trưng có hệ số dương liên quan đến lớp “tin giả”, trong khi các đặc trưng có hệ số âm liên quan đến lớp “tin thật”.

Có thể quan sát rằng nhiều từ khóa liên quan đến tin giả thường mang tính giật gân hoặc gây chú ý, ví dụ các cụm từ mang sắc thái khẳng định mạnh hoặc gây tò mò. Ngược lại, các từ khóa liên quan đến tin thật thường xuất hiện trong văn phong báo chí chính thống, chẳng hạn như các từ liên quan đến cơ quan nhà nước, tổ chức hoặc thuật ngữ chính sách.

Kết quả này cho thấy mô hình Logistic Regression không chỉ học các đặc trưng tần suất đơn thuần mà còn khai thác được những mẫu ngôn ngữ đặc trưng của từng loại tin. Điều này giúp tăng tính minh bạch của mô hình, vì các quyết định phân loại có thể được giải thích trực tiếp thông qua các trọng số đặc trưng.



Hình 4.9: Phân loại lỗi theo độ dài văn bản và mức độ tin cậy dự đoán

Hình 4.9 trình bày phân tích lỗi theo độ dài văn bản và mức độ tin cậy của dự đoán. Ở biểu đồ bên trái, có thể quan sát rằng số lượng lỗi cao hơn đối với các văn bản ngắn. Điều này có thể được giải thích bởi việc các văn bản ngắn thường thiếu ngữ cảnh và chứa ít đặc trưng TF-IDF hơn, khiến mô hình khó phân biệt giữa tin thật và tin giả.

Biểu đồ bên phải cho thấy phần lớn các lỗi xảy ra ở các dự đoán có mức độ tin cậy thấp hoặc trung bình. Trong khi đó, các dự đoán có độ tin cậy cao hiếm khi bị sai. Kết quả này cho thấy xác suất dự đoán của mô hình có tương quan tốt với độ chính xác thực tế.

Phân tích này gợi ý rằng trong các ứng dụng thực tế, những dự đoán có độ tin cậy thấp có thể được chuyển sang bước kiểm tra thủ công hoặc kết hợp với các hệ thống fact-checking để tăng độ tin cậy tổng thể của hệ thống.

4.6 Kết luận chương

Thực nghiệm toàn diện xác nhận PhoBERT là mô hình tốt nhất ($F1 = 89.88\%$) và vượt trội có ý nghĩa thống kê. Tiền huấn luyện trên ngữ liệu tiếng Việt là yếu tố quyết định, không phải kiến trúc phức tạp. SVM vượt BiLSTM là minh chứng rõ ràng cho vai trò của đặc trưng chất lượng trong học máy truyền thống.

Kết luận

Tóm tắt đóng góp

Nghiên cứu xây dựng và so sánh bốn phương pháp phát hiện tin giả tiếng Việt trên bộ dữ liệu 13,958 mẫu. Kết quả chính:

- **PhoBERT** đạt hiệu năng tốt nhất: Accuracy = 90.07%, F1 = 89.88%, AUC = 0.9495.
- Cải thiện **+6.59%** F1 so với baseline Logistic Regression.
- Kiểm định McNemar xác nhận sự vượt trội **có ý nghĩa thống kê** ($p < 0.001$).
- SVM vượt BiLSTM, minh chứng cho tầm quan trọng của chất lượng đặc trưng hơn độ phức tạp kiến trúc.
- Ablation study: tiền huấn luyện trên tiếng Việt là nhân tố quyết định; phân đoạn từ ảnh hưởng ít với TF-IDF nhưng quan trọng với PhoBERT.
- Phân tích lỗi xác định 73 mẫu khó và cho thấy tin giả hay giả mạo phong cách thông tấn quốc tế.

Hạn chế

- Tập dữ liệu có thể chưa bao phủ đầy đủ tất cả các thể loại và chiến thuật lan truyền tin giả trong thực tế, đặc biệt là các nội dung mới xuất hiện hoặc có tính chuyên biệt theo từng miền.
- Mô hình có thể suy giảm hiệu năng khi áp dụng trên dữ liệu ngoài miền huấn luyện (domain shift), đặc biệt đối với các hình thức ngụy trang tinh vi hoặc biến thể ngôn ngữ mới.
- Mô hình PhoBERT yêu cầu tài nguyên tính toán tương đối lớn, điều này có thể gây hạn chế khi triển khai trong môi trường thực tế quy mô lớn hoặc thời gian thực.

Hướng phát triển

- Thu thập thêm dữ liệu đa dạng thể loại tin giả tiếng Việt.
- Tích hợp thông tin đa phương thức (hình ảnh, đồ thị lan truyền mạng xã hội).
- Xây dựng hệ thống phát hiện thời gian thực với PhoBERT được nén nhỏ (knowledge distillation).
- Kết hợp kiểm chứng thực tế (fact-checking) với phân loại tự động.

Tài liệu tham khảo

- [1] B. Efron, “Bootstrap methods: Another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [2] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [3] D. Jurafsky and J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*, 3rd ed., 2026, online manuscript released January 6, 2026. [Online]. Available: <https://web.stanford.edu/~jurafsky/slp3/>
- [4] D. Naik and C. Jaidhar, “A novel multi-layer attention framework for visual description prediction using bidirectional lstm,” *Journal of Big Data*, vol. 9, 11 2022.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.
- [6] T. Vu, D. Q. Nguyen, D. Q. Nguyen, M. Dras, and M. Johnson, “VnCoreNLP: A vietnamese natural language processing toolkit,” in *Proceedings of NAACL-HLT 2018: Demonstrations*, 2018, pp. 56–60.
- [7] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, “Distributed representations of words and phrases and their compositionality,” in *Advances in Neural Information Processing Systems (NeurIPS 2013)*, 2013, pp. 3111–3119.
- [8] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, “Enriching word vectors with subword information,” *Transactions of the Association for Computational Linguistics*, vol. 5, pp. 135–146, 2017.
- [9] D. R. Cox, “The regression analysis of binary sequences,” *Journal of the Royal Statistical Society: Series B*, vol. 20, no. 2, pp. 215–242, 1958.
- [10] K. S. Jones, “A statistical interpretation of term specificity and its application in retrieval,” *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972.

- [11] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [12] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” in *Proceedings of the International Conference on Learning Representations (ICLR 2015)*, 2015.
- [13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, 2019, pp. 4171–4186.
- [14] D. Q. Nguyen and A. T. Nguyen, “PhoBERT: Pre-trained language models for vietnamese,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1037–1042.
- [15] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “RoBERTa: A robustly optimized BERT pretraining approach,” in *Proceedings of the International Conference on Learning Representations (ICLR 2019)*, 2019.
- [16] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL 2016)*, 2016, pp. 1715–1725.
- [17] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.
- [18] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proceedings of the International Conference on Learning Representations (ICLR 2019)*, 2019.
- [19] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [20] M. P. Naeini, G. F. Cooper, and M. Hauskrecht, “Obtaining well calibrated probabilities using bayesian binning into quantiles,” in *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence (AAAI 2015)*, 2015, pp. 2901–2907.

- [21] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [22] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.